V13_of_Final_Project_Yelp

March 15, 2026

1 Yelp Open Dataset: Leveraging Machine Learning for Smarter Restaurant Decisions

MSDS 422 Winter Quarter Final Project

By Anna Prunty-Burkart and Andrea Mendoza

[1]:
```
# Uncomment and run this cell first if any packages are missing
!pip install numpy pandas scipy matplotlib seaborn folium --quiet
!pip install nltk textblob --quiet
!pip install scikit-learn --quiet
!pip install psutil swifter tqdm --quiet
!pip install lightgbm --quiet
!pip install ipywidgets --quiet
!pip install tensorflow --quiet
!pip install tensorflow-macos tensorflow-metal --quiet
```

```
ERROR: Could not find a version that satisfies the requirement tensorflow-
macos (from versions: none)
ERROR: No matching distribution found for tensorflow-macos
```

[2]:
```
# ================================
# Core Python
# ================================
import os
import re
import time
import itertools
from pathlib import Path
from datetime import datetime, timedelta
from collections import Counter, defaultdict

# ================================
# Scientific & Data
# ================================
import numpy as np
```

```python
import pandas as pd
from scipy import stats
from scipy.stats import ttest_ind

# ================================
# Visualization
# ================================
import matplotlib.pyplot as plt
import seaborn as sns
import folium

# ================================
# NLP
# ================================
import nltk
from nltk.corpus import stopwords
from nltk import bigrams
from textblob import TextBlob

# ================================
# Machine Learning (Sklearn)
# ================================
from sklearn.model_selection import train_test_split, cross_val_score,␣
↪StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.linear_model import SGDClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# ================================
# Deep Learning (TensorFlow / Keras)
# ================================
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping

# ================================
# Utilities
# ================================
import psutil
import swifter
```

```
from tqdm.auto import tqdm
from tqdm import tqdm as tqdm_text
tqdm.pandas()
```

1.1 Load and join datasets

For this analysis, we loaded the Yelp Business and Reviews datasets and created a merged dataset by combining aggregated review data with business attributes. To focus our analysis and improve processing efficiency, we loaded in solely businesses categorized as "Restaurants" and filtered reviews to only those restaurant business and only those from January 1, 2018 to 2022. To clean our dataset, we removed duplicates from the Reviews dataset and entries from the Restaurants dataset that did not have an average review rating.

[3]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

[5]:
```
# Paths to datasets
biz_path = '/content/drive/MyDrive/MSDS 422_Final Project_Yelp/
 ↪yelp_academic_dataset_business.json'
rev_path = '/content/drive/MyDrive/MSDS 422_Final Project_Yelp/
 ↪yelp_academic_dataset_review.json'

# Load business data in chunks to find restaurant business IDs
restaurant_ids = set()

for chunk in pd.read_json(biz_path, lines=True, chunksize=10000):
    # Filter businesses with "Restaurants" in categories
    chunk = chunk[chunk["categories"].str.contains("Restaurants", na=False)]
    restaurant_ids.update(chunk["business_id"].unique())

print(f"Found {len(restaurant_ids)} restaurant business IDs.")
```

```
Found 52268 restaurant business IDs.
```

[6]:
```
# Load all columns for restaurant businesses
biz_chunks = []

for chunk in pd.read_json(biz_path, lines=True, chunksize=10000):
    # Filter for restaurants
    chunk = chunk[chunk['business_id'].isin(restaurant_ids)]

    if not chunk.empty:
        # Flatten attributes if the column exists
        if 'attributes' in chunk.columns:
            chunk['attributes'] = chunk['attributes'].apply(
```

```
                lambda x: eval(str(x)) if pd.notna(x) else {}
            )
            attrs = pd.json_normalize(chunk['attributes'])
            chunk = pd.concat([chunk.drop(columns=['attributes']), attrs],␣
 ↪axis=1)
        biz_chunks.append(chunk)

business_df = pd.concat(biz_chunks, ignore_index=True)
print(f"Filtered Business shape: {business_df.shape}")
```

```
Filtered Business shape: (103341, 52)
```

[7]:
```
# Load and filter reviews
cutoff = '2018-01-01'
review_chunks = []

for chunk in pd.read_json(rev_path, lines=True, chunksize=100000):
    # Convert date to datetime and filter
    chunk['date'] = pd.to_datetime(chunk['date'])
    chunk = chunk[chunk['business_id'].isin(restaurant_ids) & (chunk['date'] >=␣
 ↪cutoff)]

    if not chunk.empty:
        review_chunks.append(chunk)

reviews_df = pd.concat(review_chunks, ignore_index=True)
print(f"Filtered Reviews shape: {reviews_df.shape}")
```

```
Filtered Reviews shape: (2007019, 9)
```

[8]:
```
# Check for duplicate reviews (same review_id or same␣
 ↪user_id+business_id+text+date)
review_duplicates = reviews_df[
    reviews_df.duplicated(subset=['review_id'], keep=False)  # review_id should␣
 ↪be unique
    | reviews_df.duplicated(subset=['user_id', 'business_id', 'text', 'date'],␣
 ↪keep=False)  # Natural key
]

print(f"Duplicate reviews found: {len(review_duplicates)}")
if not review_duplicates.empty:
    print("Sample duplicates:\n", review_duplicates.head())
    # Drop duplicates (keep first occurrence)
    reviews_df = reviews_df.drop_duplicates(subset=['review_id'], keep='first')
```

```
Duplicate reviews found: 4
Sample duplicates:
                       review_id                 user_id  \
```

```
767934   CB9ScDB-MYTWVyd1E9dSgw  wAph_egpAXr9ic-A2vnINQ
776074   qcYQmZuP4ywuYmYfcyAuYQ  wAph_egpAXr9ic-A2vnINQ
1543500  XjlaQ5ImGg1uL2XelQA2xw  xZ0ioQJ0Jyo_Uc0FzmyFQQ
1567124  8JcvaBpFPAe7lIVSazi0hQ  xZ0ioQJ0Jyo_Uc0FzmyFQQ

                    business_id  stars  useful  funny  cool  \
767934   fiGh8ctcvIRm4Ii2GOtoHQ      2       0      0     0
776074   fiGh8ctcvIRm4Ii2GOtoHQ      2       0      0     0
1543500  O_kAd89gPls-dXda40NWzA      5       0      0     0
1567124  O_kAd89gPls-dXda40NWzA      5       0      0     0

                                                     text                 date
767934   We have been to Trax several times before, wit… 2018-07-23 21:43:33
776074   We have been to Trax several times before, wit… 2018-07-23 21:43:33
1543500  My son and I moved here from South Carolina a … 2019-11-22 07:18:46
1567124  My son and I moved here from South Carolina a … 2019-11-22 07:18:46
```

[9]:
```python
# Aggregate review data
review_agg_df = (
    reviews_df
    .groupby('business_id')
    .agg(
        review_count=('stars', 'count'),
        avg_rating=('stars', 'mean'),
        rating_std=('stars', 'std')
    )
    .reset_index()
)

# Merge with business data (keep all businesses initially)
restaurant_df = business_df.merge(review_agg_df, on='business_id', how='left')

# Drop businesses with no reviews (missing avg_rating)
restaurant_df = restaurant_df.dropna(subset=['avg_rating']).copy()

# Verify
print(f"Businesses with missing avg_rating dropped: {len(business_df) -␣
↪len(restaurant_df)}")
print(f"Remaining businesses: {len(restaurant_df)}")
```

```
Businesses with missing avg_rating dropped: 61067
Remaining businesses: 42274
```

1.2 Conduct and refine exploratory data analysis (EDA)

[10]:
```
# Display results
print("\nSample of Restaurants-Only Reviews Data:")
display(reviews_df.head(3))

print("\nSample of Resaurants-Only Business Data:")
display(business_df.head(3))

print("\nSample of Merged Restaurant Business & Review Data:")
display(restaurant_df.head(3))
```

```
Sample of Restaurants-Only Reviews Data:

                review_id                 user_id             business_id  \
0  KU_O5udG6zpxOg-VcAEodg  mh_-eMZ6K5RLWhZyISBhwA  XQfwVwDr-v0ZS3_CbbE5Xw
1  lUUhg8ltDsUZ9h0xnwY4Dg  RreNy--tOmXMl1en0wiBOg  cPepkJeRMtHapc_b2Oe_dw
2  JBWZmBy69VMggxj3eYn17Q  aFa96pz67TwOFu4Weq5Agg  kq5Ghhh14r-eCxlVmlyd8w

   stars  useful  funny  cool  \
0      3       0      0     0
1      4       1      0     1
2      5       0      0     0

                                                text                date
0  If you decide to eat here, just be aware it is… 2018-07-07 22:09:11
1  I was really between 3 and 4 stars for this on… 2018-07-17 03:30:07
2  My boyfriend and I tried this deli for the fir… 2018-08-23 21:39:38


Sample of Resaurants-Only Business Data:

              business_id                   name            address  \
0  MTSW4McQd7CbVtyjqoe9mw     St Honore Pastries        935 Race St
1  CF33F8-E6oudUQ46HnavjQ         Sonic Drive-In      615 S Main St
2  k0hlBqXX-Bt0vf1op7Jr1w  Tsevi's Pub And Grill  8025 Mackenzie Rd

           city state postal_code   latitude  longitude  stars  review_count  \
0  Philadelphia    PA       19107  39.955505 -75.155564    4.0          80.0
1  Ashland City    TN       37015  36.269593 -87.058943    2.0           6.0
2        Affton    MO       63123  38.565165 -90.321087    3.0          19.0

   …  BestNights BYOB Corkage BYOBCorkage RestaurantsCounterService  \
0  …         NaN  NaN     NaN         NaN                       NaN
1  …         NaN  NaN     NaN         NaN                       NaN
2  …         NaN  NaN     NaN         NaN                       NaN

  Open24Hours AgesAllowed DietaryRestrictions HairSpecializesIn  \
```

```
0         NaN         NaN                 NaN               NaN
1         NaN         NaN                 NaN               NaN
2         NaN         NaN                 NaN               NaN

  AcceptsInsurance
0              NaN
1              NaN
2              NaN

[3 rows x 52 columns]


Sample of Merged Restaurant Business & Review Data:
              business_id                name              address  \
0  MTSW4McQd7CbVtyjqoe9mw  St Honore Pastries          935 Race St
1  CF33F8-E6oudUQ46HnavjQ      Sonic Drive-In        615 S Main St
3  bBDDEgkFA1Otx9Lfe7BZUQ      Sonic Drive-In  2312 Dickerson Pike

           city state postal_code   latitude  longitude  stars  \
0  Philadelphia    PA       19107  39.955505 -75.155564    4.0
1  Ashland City    TN       37015  36.269593 -87.058943    2.0
3     Nashville    TN       37207  36.208102 -86.768170    1.5

   review_count_x  …  BYOBCorkage RestaurantsCounterService Open24Hours  \
0            80.0  …          NaN                       NaN         NaN
1             6.0  …          NaN                       NaN         NaN
3            10.0  …          NaN                       NaN         NaN

  AgesAllowed DietaryRestrictions HairSpecializesIn AcceptsInsurance  \
0         NaN                 NaN               NaN              NaN
1         NaN                 NaN               NaN              NaN
3         NaN                 NaN               NaN              NaN

  review_count_y avg_rating rating_std
0           30.0   4.466667   1.136642
1            4.0   1.250000   0.500000
3            6.0   1.666667   1.632993

[3 rows x 55 columns]
```

[11]:
```python
# Shape, column list, dtypes
print("Rows / Columns :", restaurant_df.shape)
print("\nColumns and dtypes:")
display(restaurant_df.dtypes)

# Missing-value summary
missing = restaurant_df.isna().sum()
```

```python
missing = missing[missing > 0].sort_values(ascending=False)
print("\nColumns with missing values (if any):")
display(missing)

# Numeric summary
numeric_cols = restaurant_df.select_dtypes(include=['int64','float64']).columns
display(restaurant_df[numeric_cols].describe().T)
```

```
Rows / Columns : (42274, 55)

Columns and dtypes:
business_id                    object
name                           object
address                        object
city                           object
state                          object
postal_code                    object
latitude                      float64
longitude                     float64
stars                         float64
review_count_x                float64
is_open                       float64
categories                     object
hours                          object
RestaurantsDelivery            object
OutdoorSeating                 object
BusinessAcceptsCreditCards     object
BusinessParking                object
BikeParking                    object
RestaurantsPriceRange2         object
RestaurantsTakeOut             object
ByAppointmentOnly              object
WiFi                           object
Alcohol                        object
Caters                         object
RestaurantsAttire              object
RestaurantsReservations        object
Ambience                       object
GoodForKids                    object
CoatCheck                      object
DogsAllowed                    object
RestaurantsTableService        object
RestaurantsGoodForGroups       object
WheelchairAccessible           object
HasTV                          object
HappyHour                      object
DriveThru                      object
```

```
NoiseLevel                    object
GoodForMeal                   object
BusinessAcceptsBitcoin        object
Smoking                       object
Music                         object
GoodForDancing                object
BestNights                    object
BYOB                          object
Corkage                       object
BYOBCorkage                   object
RestaurantsCounterService     object
Open24Hours                   object
AgesAllowed                   object
DietaryRestrictions           object
HairSpecializesIn             object
AcceptsInsurance              object
review_count_y               float64
avg_rating                   float64
rating_std                   float64
dtype: object


Columns with missing values (if any):

AcceptsInsurance             42274
RestaurantsCounterService    42274
HairSpecializesIn            42274
Open24Hours                  42274
DietaryRestrictions          42274
AgesAllowed                  42273
BYOBCorkage                  42251
Smoking                      42211
Corkage                      42210
BYOB                         42206
ByAppointmentOnly            42205
GoodForDancing               42203
CoatCheck                    42198
BestNights                   42188
Music                        42173
BusinessAcceptsBitcoin       42161
DriveThru                    42144
DogsAllowed                  42049
WheelchairAccessible         42042
HappyHour                    42034
RestaurantsTableService      41906
GoodForMeal                  41737
Caters                       41649
NoiseLevel                   41632
BikeParking                  41630
```

```
WiFi                          41584
RestaurantsAttire             41558
Alcohol                       41543
GoodForKids                   41529
Ambience                      41522
RestaurantsGoodForGroups      41510
HasTV                         41510
RestaurantsReservations       41487
OutdoorSeating                41485
BusinessAcceptsCreditCards    41462
RestaurantsPriceRange2        41451
BusinessParking               41441
RestaurantsDelivery           41405
RestaurantsTakeOut            41387
hours                          3988
rating_std                     1474
dtype: int64

                  count        mean         std         min        25%  \
latitude        42274.0   36.980009    6.048417   27.564457  32.192566
longitude       42274.0  -87.834563   13.718598 -120.083748 -90.229115
stars           42274.0    3.539303    0.847906    1.000000   3.000000
review_count_x  42274.0  100.581823  206.725056    5.000000  16.000000
is_open         42274.0    0.819676    0.384461    0.000000   1.000000
review_count_y  42274.0   47.476439   93.622664    1.000000   8.000000
avg_rating      42274.0    3.480349    0.987305    1.000000   2.875000
rating_std      40800.0    1.312276    0.455231    0.000000   1.095445

                      50%         75%          max
latitude        39.472986   39.958950    53.679197
longitude      -86.041920  -75.361143   -74.664459
stars            3.500000    4.000000     5.000000
review_count_x  41.000000  107.000000  7568.000000
is_open          1.000000    1.000000     1.000000
review_count_y  20.000000   51.000000  4226.000000
avg_rating       3.690476    4.227723     5.000000
rating_std       1.395455    1.602082     2.828427
```

[12]:
```python
# Check for duplicate businesses (same business_id)
restaurant_duplicates = restaurant_df[
    restaurant_df.duplicated(subset=['business_id'], keep=False)
]

print(f"\nDuplicate restaurants found: {len(restaurant_duplicates)}")
if not restaurant_duplicates.empty:
    print("Sample duplicates:\n", restaurant_duplicates[['business_id', 'name',␣
 ↪'city']].head())
    # Drop duplicates (keep first occurrence)
```

```
    restaurant_df = restaurant_df.drop_duplicates(subset=['business_id'],␣
↪keep='first')
```

```
Duplicate restaurants found: 0
```

[13]:
```
# Shape, column list, dtypes
print("Rows / Columns :", reviews_df.shape)
print("\nColumns and dtypes:")
display(reviews_df.dtypes)

# Missing-value summary
missing = reviews_df.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
print("\nColumns with missing values (if any):")
display(missing)

# Numeric summary
numeric_cols = reviews_df.select_dtypes(include=['int64','float64']).columns
display(reviews_df[numeric_cols].describe().T)
```

```
Rows / Columns : (2007019, 9)

Columns and dtypes:

review_id              object
user_id                object
business_id            object
stars                   int64
useful                  int64
funny                   int64
cool                    int64
text                   object
date           datetime64[ns]
dtype: object


Columns with missing values (if any):

Series([], dtype: int64)

            count      mean       std  min  25%  50%  75%    max
stars   2007019.0  3.842482  1.462622  1.0  3.0  5.0  5.0    5.0
useful  2007019.0  0.860922  2.801029  0.0  0.0  0.0  1.0  325.0
funny   2007019.0  0.223364  1.373725  0.0  0.0  0.0  0.0  254.0
cool    2007019.0  0.483623  2.364750  0.0  0.0  0.0  0.0  304.0
```

We examined the secondary categories associated with businesses categorized as "Restaurants" (e.g., "Burgers," "Fast Food," "Sandwiches"). We created a dedicated column capturing these other categories.

[14]:
```
# Initialize counter
additional_category_counts = Counter()

# Loop through raw 'categories' column
for categories in restaurant_df['categories']:

    if isinstance(categories, str):

        # Split string into list and clean whitespace
        category_list = [cat.strip() for cat in categories.split(',')]

        # Remove 'Restaurants'
        other_categories = [cat for cat in category_list if cat !=␣
 ↪'Restaurants']

        additional_category_counts.update(other_categories)

# Display results
print("\nTop 30 most frequent additional categories for businesses with␣
 ↪'Restaurants':")
for category, count in additional_category_counts.most_common(30):
    print(f"{category}: {count}")
```

```
Top 30 most frequent additional categories for businesses with 'Restaurants':
Food: 13026
Sandwiches: 7267
Nightlife: 7252
Bars: 6992
American (Traditional): 6824
Fast Food: 6102
Pizza: 6018
Breakfast & Brunch: 5502
Burgers: 5030
American (New): 4699
Mexican: 3880
Italian: 3625
Coffee & Tea: 3560
Seafood: 3045
Salad: 2796
Chicken Wings: 2722
Event Planning & Services: 2687
Chinese: 2662
Cafes: 2232
Delis: 2019
Caterers: 1792
Specialty Food: 1637
Desserts: 1624
```

```
Bakeries: 1579
Sports Bars: 1533
Japanese: 1466
Sushi Bars: 1373
Cocktail Bars: 1312
Barbeque: 1292
Steakhouses: 1255
```

[15]:
```python
restaurant_df['secondary_category'] = restaurant_df['categories'].apply(
    lambda x: [
        cat.strip()
        for cat in x.split(',')
        if cat.strip().lower() != 'restaurants'
    ] if isinstance(x, str) else []
)

restaurant_df[['categories', 'secondary_category']].head()
```

[15]:
```
                                          categories  \
0  Restaurants, Food, Bubble Tea, Coffee & Tea, B…
1  Burgers, Fast Food, Sandwiches, Food, Ice Crea…
3  Ice Cream & Frozen Yogurt, Fast Food, Burgers,…
4         Vietnamese, Food, Restaurants, Food Trucks
5  American (Traditional), Restaurants, Diners, B…

                                  secondary_category
0         [Food, Bubble Tea, Coffee & Tea, Bakeries]
1  [Burgers, Fast Food, Sandwiches, Food, Ice Cre…
3  [Ice Cream & Frozen Yogurt, Fast Food, Burgers…
4                    [Vietnamese, Food, Food Trucks]
5  [American (Traditional), Diners, Breakfast & B…
```

By examining the discrepancy between the stars rating in the business dataset and the average rating computed from the reviews dataset, we determined that the stars variable should be dropped from the cleaned dataset. The business-level stars rating represents a static snapshot, whereas the review-based average rating is dynamically derived from individual reviews and provides a more informative feature.

[16]:
```python
# Examine the discrepancy between the business' snapshot/static "stars" rating
# and the average rating from all reviews (sourced from REVIEWS dataset)
diff = (restaurant_df['avg_rating'] - restaurant_df['stars']).abs().max()
print(f'Max absolute difference between avg_rating and stars: {diff:.6f}')
```

```
Max absolute difference between avg_rating and stars: 3.500000
```

[17]:
```python
# Drop 'stars' column from restaurant_df
restaurant_df = restaurant_df.drop(columns=['stars'])

# Check that column was dropped
```

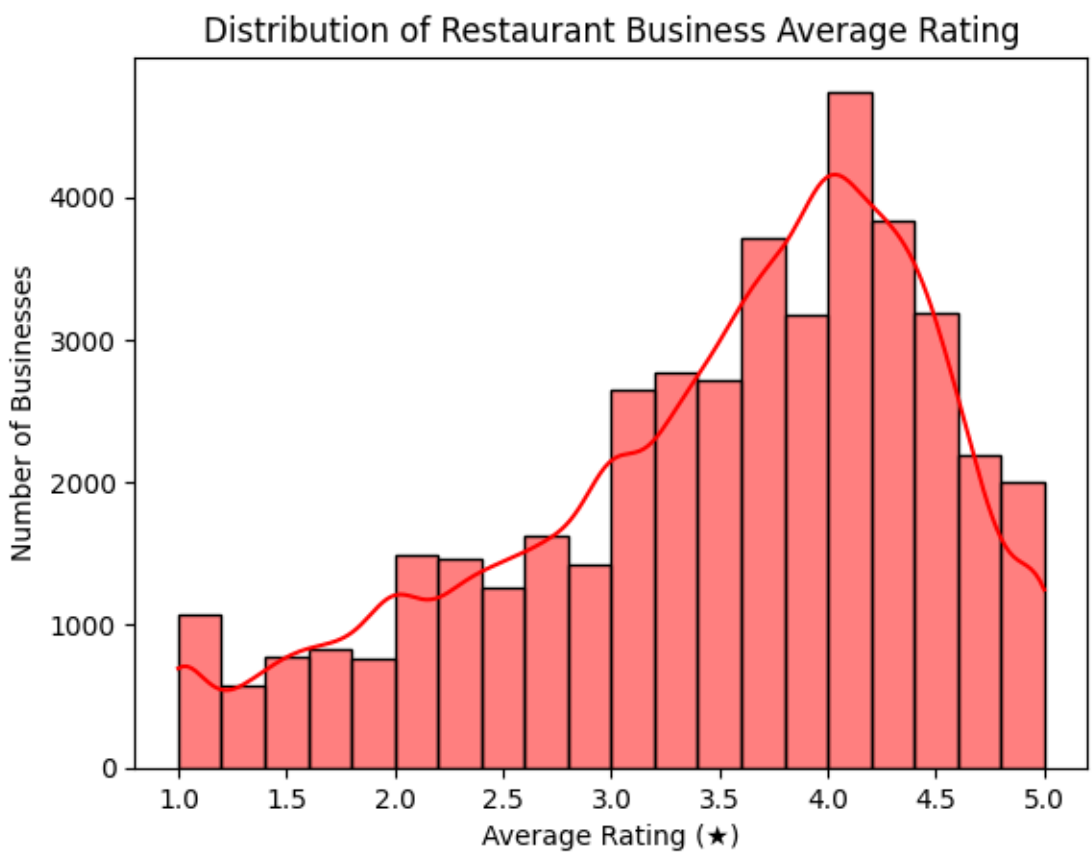
```python
print(restaurant_df.columns)
```

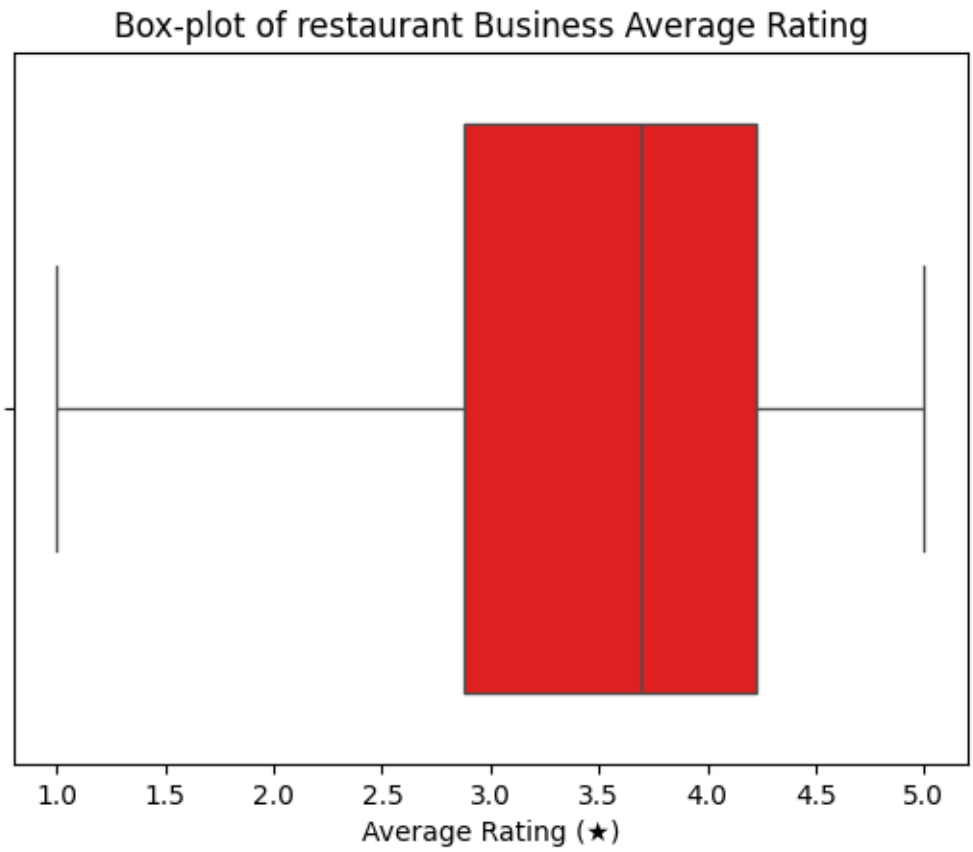
```
Index(['business_id', 'name', 'address', 'city', 'state', 'postal_code',
       'latitude', 'longitude', 'review_count_x', 'is_open', 'categories',
       'hours', 'RestaurantsDelivery', 'OutdoorSeating',
       'BusinessAcceptsCreditCards', 'BusinessParking', 'BikeParking',
       'RestaurantsPriceRange2', 'RestaurantsTakeOut', 'ByAppointmentOnly',
       'WiFi', 'Alcohol', 'Caters', 'RestaurantsAttire',
       'RestaurantsReservations', 'Ambience', 'GoodForKids', 'CoatCheck',
       'DogsAllowed', 'RestaurantsTableService', 'RestaurantsGoodForGroups',
       'WheelchairAccessible', 'HasTV', 'HappyHour', 'DriveThru', 'NoiseLevel',
       'GoodForMeal', 'BusinessAcceptsBitcoin', 'Smoking', 'Music',
       'GoodForDancing', 'BestNights', 'BYOB', 'Corkage', 'BYOBCorkage',
       'RestaurantsCounterService', 'Open24Hours', 'AgesAllowed',
       'DietaryRestrictions', 'HairSpecializesIn', 'AcceptsInsurance',
       'review_count_y', 'avg_rating', 'rating_std', 'secondary_category'],
      dtype='object')
```

By visualizing the target variable, average rating per business, it's clear the ratings have a positive skewness with a left tail.

[18]:
```python
# Examine Target Variable: Average Ratings

target = 'avg_rating' # the column we'll predict

# Histogram (unweighted)
sns.histplot(restaurant_df[target], bins=20, kde=True, color='red')
plt.title('Distribution of Restaurant Business Average Rating')
plt.xlabel('Average Rating ( )')
plt.ylabel('Number of Businesses')
plt.show()


# Box-plot (detect outliers / extreme values)

sns.boxplot(x=restaurant_df[target], color='red')
plt.title('Box-plot of restaurant Business Average Rating')
plt.xlabel('Average Rating ( )')
plt.show()


# Basic descriptive stats

stats = restaurant_df[target].describe(percentiles=[.05, .25, .5, .75, .95])
print(stats)


# Weighted overall rating (for reference)
```

```
weighted_mean = (restaurant_df[target] * restaurant_df['review_count_y']).sum()␣
↪/ restaurant_df['review_count_y'].sum()
print(f"\nWeighted overall rating (all reviews): {weighted_mean:.3f}")
```

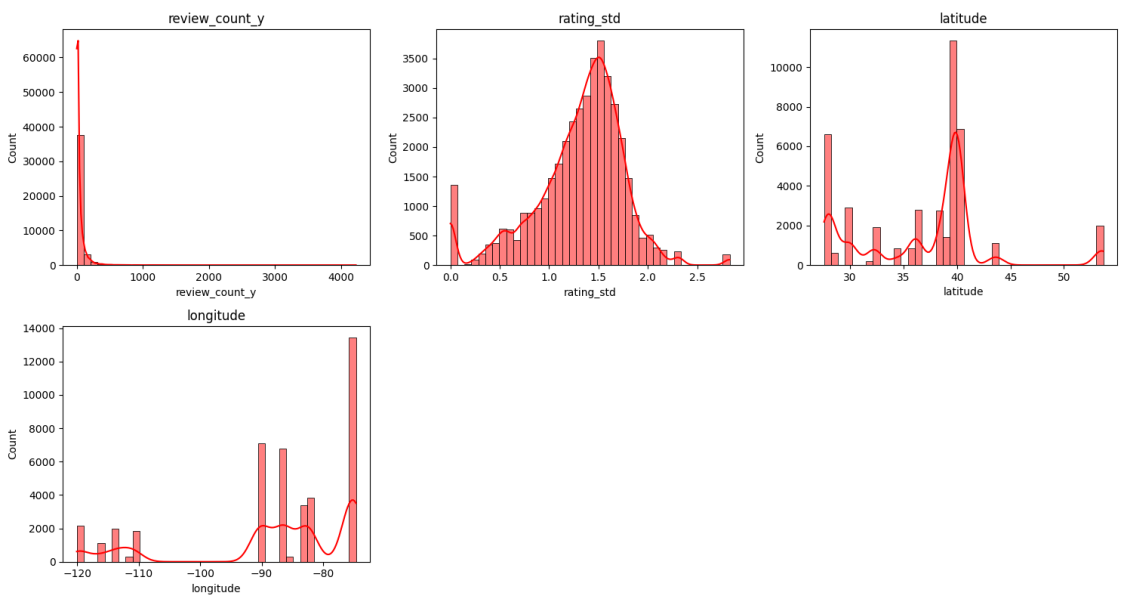





```
count    42274.000000
mean         3.480349
std          0.987305
min          1.000000
5%           1.500000
25%          2.875000
50%          3.690476
75%          4.227723
95%          4.800000
max          5.000000
Name: avg_rating, dtype: float64

Weighted overall rating (all reviews): 3.842
```

[19]:
```
# Plot histograms for a list of columns

def plot_numeric(cols, ncol=3, bins=30):
    nrow = int(np.ceil(len(cols) / ncol))
    fig, axes = plt.subplots(nrow, ncol, figsize=(ncol*5, nrow*4))
    axes = axes.ravel()
```

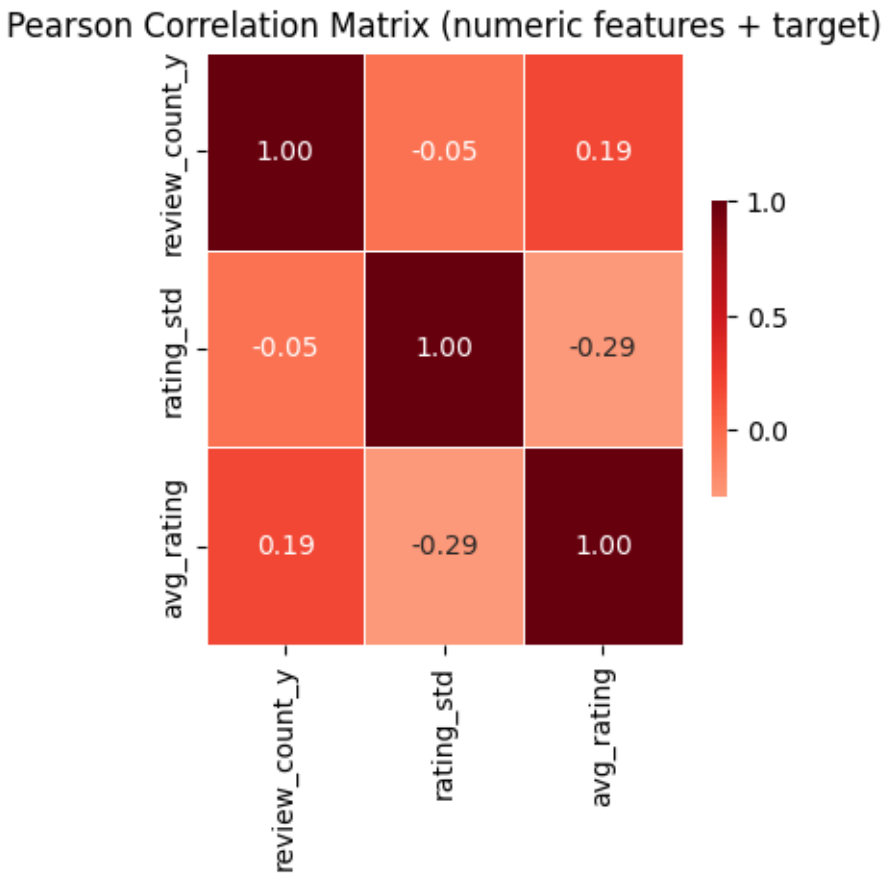
```
    for i, col in enumerate(cols):
        sns.histplot(restaurant_df[col].dropna(), bins=bins, kde=True,␣
↪ax=axes[i], color='red')
        axes[i].set_title(col)
    # hide any unused sub-plots
    for j in range(i+1, len(axes)):
        axes[j].axis('off')
    plt.tight_layout()
    plt.show()

numeric_features = [
    'review_count_y', 'rating_std','latitude', 'longitude'
]

plot_numeric(numeric_features, ncol=3, bins=40)
```



In examining the correlatino heatmap, there is a moderate negative correlation between average rating and rating standard deviation, suggesting that businesses with greater variability in their ratings tend to have lower average ratings.

[20]:
```
# Pearson correlation heatmap

# Exclude latitude and longitude
features_for_corr = [
    col for col in numeric_features
    if col not in ["latitude", "longitude"]
]
```

```python
corr = restaurant_df[features_for_corr + [target]].corr()

# Show the full matrix
plt.figure(figsize=(4,4))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="Reds",
    center=0,
    linewidths=.5,
    cbar_kws={"shrink": .5}
)
plt.title("Pearson Correlation Matrix (numeric features + target)")
plt.show()

# Top features most correlated with the target
k = 5
top_corr = (
    corr[target]
    .abs()
    .sort_values(ascending=False)
    .drop(labels=[target])
    .head(k)
)

print(f"\nTop {k} features most correlated with {target}:")
display(top_corr)
```




```
Top 5 features most correlated with avg_rating:

rating_std        0.290426
review_count_y    0.186005
Name: avg_rating, dtype: float64
```

We examined the distribution of average star rating across the cities and states with top and lowest ratings to investigate potential correlation with the target variable.

[21]:
```
sns.set_style('whitegrid')

# --- City-level aggregation (30 businesses) ---
city_stats = (
    restaurant_df.groupby('city')
    .agg(
        businesses=('business_id', 'nunique'),
        avg_rating=('avg_rating', 'mean')
    )
    .query('businesses >= 30')
```

```
)

# --- Top & Bottom 5 ---
top5_cities    = city_stats.nlargest(5, 'avg_rating')
bottom5_cities = city_stats.nsmallest(5, 'avg_rating')

print("Top 5 cities ( 30 businesses) by average rating:")
display(top5_cities)

print("\nBottom 5 cities ( 30 businesses) by average rating:")
display(bottom5_cities)


# --- Plot function ---
def barplot_city(df, title, color, xmax=None):
    plt.figure(figsize=(7, 4))  # smaller figure

    ordered = df['avg_rating'].sort_values(ascending=False)

    ax = ordered.plot(
        kind='barh',
        color=color,
        edgecolor='black'
    )

    ax.set_xlabel('Average Rating')
    ax.set_ylabel('City')
    ax.set_title(title, fontsize=11)

    if xmax is not None:
        ax.set_xlim(0, xmax)

    # Add value labels
    for i, v in enumerate(ordered):
        ax.text(v + 0.01, i, f"{v:.2f}", va='center', fontsize=9)

    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()


# --- Top 5 (normal scale) ---
barplot_city(
    top5_cities,
    title='Top 5 Cities ( 30 businesses) - Highest Average Rating',
    color='red',
    xmax = 5
```

```
)

# --- Bottom 5 (zoomed to 3.5) ---
barplot_city(
    bottom5_cities,
    title='Bottom 5 Cities ( 30 businesses) - Lowest Average Rating',
    color='whitesmoke',
    xmax=5
)
```

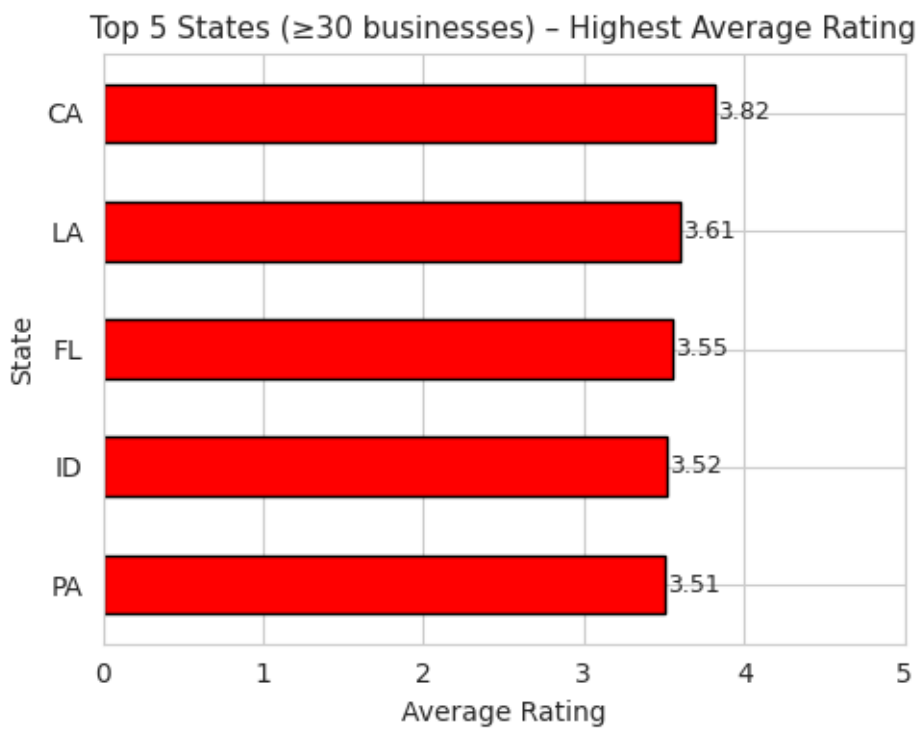
```
Top 5 cities ( 30 businesses) by average rating:

                    businesses  avg_rating
city
Lambertville                46    4.013240
Safety Harbor               42    4.001253
Gulfport                    46    3.967858
Hatboro                     46    3.933584
Indian Rocks Beach          31    3.911221


Bottom 5 cities ( 30 businesses) by average rating:

                 businesses  avg_rating
city
King Of Prussia          36    2.752506
Florissant              144    2.868744
Madison                  79    2.899712
Hazelwood                33    2.911816
Willingboro              30    2.921082
```

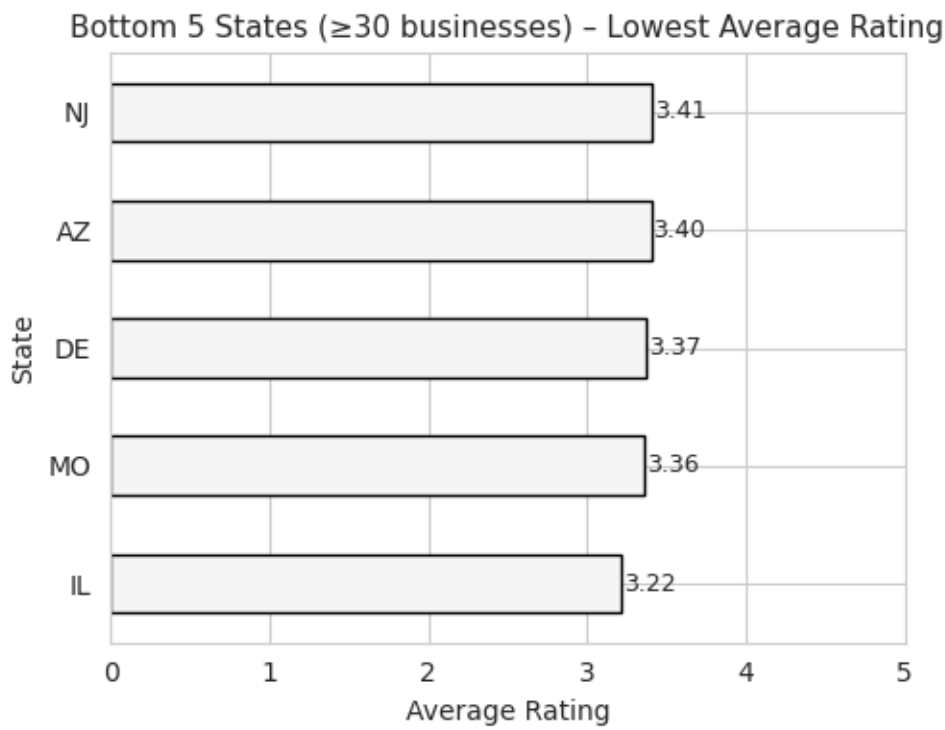
[22]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style('whitegrid')

# --- State-level aggregation (30 businesses) ---
state_stats = (
    restaurant_df.groupby('state')
    .agg(
        businesses=('business_id', 'nunique'),
        avg_rating=('avg_rating', 'mean')
    )
    .query('businesses >= 30')
)

# --- Top & Bottom 5 ---
top5_states    = state_stats.nlargest(5, 'avg_rating')
bottom5_states = state_stats.nsmallest(5, 'avg_rating')

print("Top 5 states (30 businesses) by average rating:")
display(top5_states)

print("\nBottom 5 states (30 businesses) by average rating:")
display(bottom5_states)
```

```python
# --- Plot function ---
def barplot_state(df, title, color, xmax=None):
    plt.figure(figsize=(5, 4))  # smaller figure

    ordered = df['avg_rating'].sort_values(ascending=False)

    ax = ordered.plot(
        kind='barh',
        color=color,
        edgecolor='black'
    )

    ax.set_xlabel('Average Rating')
    ax.set_ylabel('State')
    ax.set_title(title, fontsize=11)

    if xmax is not None:
        ax.set_xlim(0, xmax)

    # Add value labels
    for i, v in enumerate(ordered):
        ax.text(v + 0.01, i, f"{v:.2f}", va='center', fontsize=9)

    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()


# --- Top 5 (set scale to 5) ---
barplot_state(
    top5_states,
    title='Top 5 States ( 30 businesses) - Highest Average Rating',
    color='red',
    xmax=5
)

# --- Bottom 5 (set scale to 5) ---
barplot_state(
    bottom5_states,
    title='Bottom 5 States ( 30 businesses) - Lowest Average Rating',
    color='whitesmoke',
    xmax=5
)
```

```
Top 5 states ( 30 businesses) by average rating:
```

```
       businesses  avg_rating
state
CA            863    3.819185
LA           2911    3.606824
FL           7221    3.553158
ID           1099    3.518371
PA           9849    3.505307


Bottom 5 states ( 30 businesses) by average rating:

       businesses  avg_rating
state
IL            844    3.218707
MO           3351    3.361422
DE            816    3.370636
AZ           2134    3.402973
NJ           2787    3.406703
```





[23]:
```
import folium
import branca.colormap as cm

# Drop rows with missing latitude, longitude, or avg_rating
restaurant_df = restaurant_df.dropna(subset=['latitude', 'longitude',␣
 ↪'avg_rating'])

# Create the map
m = folium.Map()

# Create colormap with sorted thresholds
colormap = cm.linear.viridis.scale(
    restaurant_df['avg_rating'].min(),
    restaurant_df['avg_rating'].max()
)
colormap.caption = 'Average Rating'

# Add markers
for _, row in restaurant_df.iterrows():
    folium.CircleMarker(
        location=[row['latitude'], row['longitude']],
        radius=4,
        fill=True,
```

```
        fill_color=colormap(row['avg_rating']),
        color=colormap(row['avg_rating']),
        fill_opacity=0.9,
        popup=f"Rating: {row['avg_rating']:.1f}"  # Optional: Add popup with␣
 ↪rating
    ).add_to(m)

# Automatically zoom to fit all data points
bounds = [
    [restaurant_df['latitude'].min(), restaurant_df['longitude'].min()],
    [restaurant_df['latitude'].max(), restaurant_df['longitude'].max()]
]
m.fit_bounds(bounds)

# Add the colormap to the map
colormap.add_to(m)

# Display the map
m
```

[23]: <folium.folium.Map at 0x78221573deb0>

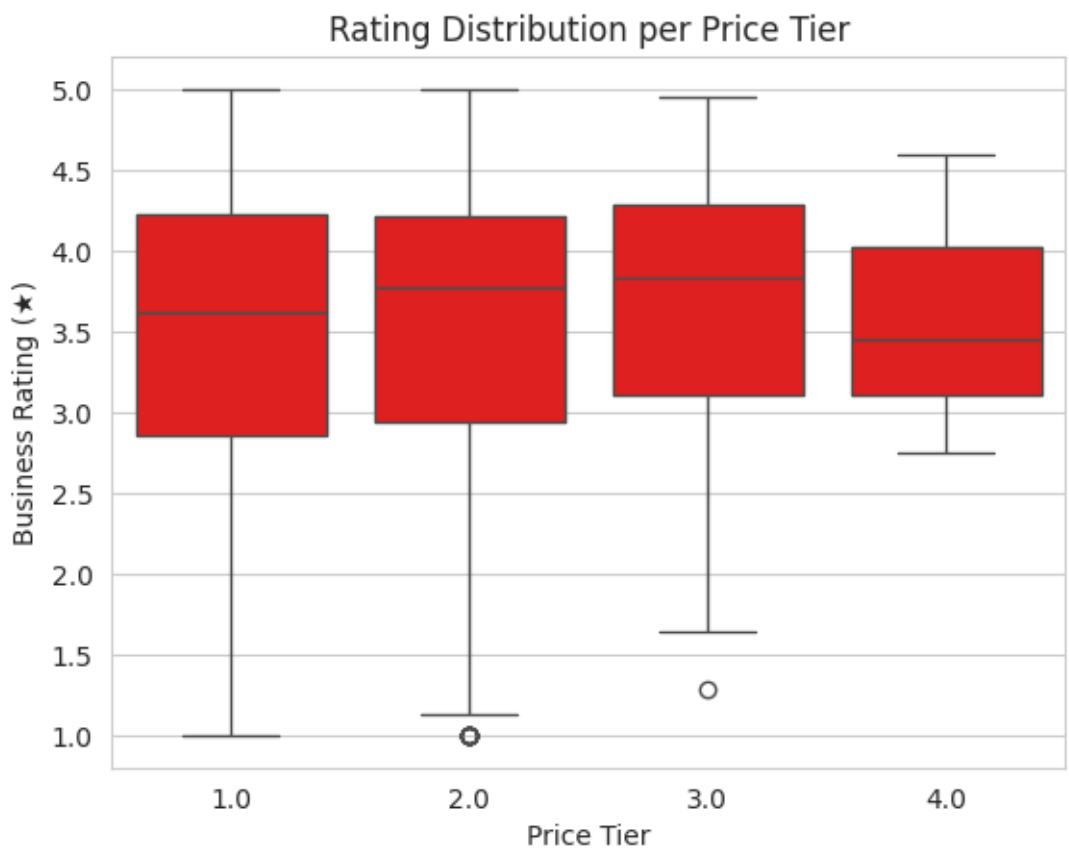
We explored the relationship between business' price range (i.e., how expensive or cheap the business is) and the average rating. The correlation was insignificant.

[24]:
```
# Examine Price-range (RestaurantsPriceRange2) vs. the average business rating␣
 ↪(avg_rating)
# Clean / convert the price-range column
price_map = {
    '1': 1,
    '2': 2,
    '3': 3,
    '4': 4,
    'None': np.nan,
    np.nan: np.nan  # Handle NaN values explicitly
}

# Create a clean numeric column
restaurant_df['price_tier'] = restaurant_df['RestaurantsPriceRange2'].
 ↪map(price_map)

# Aggregate - mean rating per price tier
price_stats = (
    restaurant_df
    .groupby('price_tier')
    .agg(
        business_cnt=('business_id','nunique'),
```

```
        avg_rating=('avg_rating','mean')
    )
    .reset_index()
    .sort_values('price_tier')
)

print("Mean rating per price tier:")
display(price_stats)

# Bar chart - average rating by price tier

sns.barplot(x='price_tier', y='avg_rating', data=price_stats,
            color='red')
plt.title('Average Business Rating by Price Tier')
plt.xlabel('Price Tier (1 = cheap, 4 = expensive)')
plt.ylabel('Average Rating ( )')
plt.ylim(0,5)
for i, row in price_stats.iterrows():
    plt.text(i, row['avg_rating']+0.05,
             f"{row['avg_rating']:.2f}", ha='center')
plt.show()

# Box-plot - rating distribution inside each tier
sns.boxplot(x='price_tier', y='avg_rating', data=restaurant_df,
            color = 'red')
plt.title('Rating Distribution per Price Tier')
plt.xlabel('Price Tier')
plt.ylabel('Business Rating ( )')
plt.show()

# Correlation
corr = restaurant_df[['price_tier','avg_rating']].dropna().corr().iloc[0,1]
print(f"Pearson correlation between price tier and rating: {corr:.3f}")
```

```
Mean rating per price tier:

   price_tier  business_cnt  avg_rating
0         1.0           367    3.455654
1         2.0           422    3.495501
2         3.0            31    3.629462
3         4.0             3    3.601515
```

Average Business Rating by Price Tier
Average Rating (★)
5
4
3
2
1
0
3.46
3.50
3.63
3.60
1.0
2.0
3.0
4.0
Price Tier (1 = cheap, 4 = expensive)



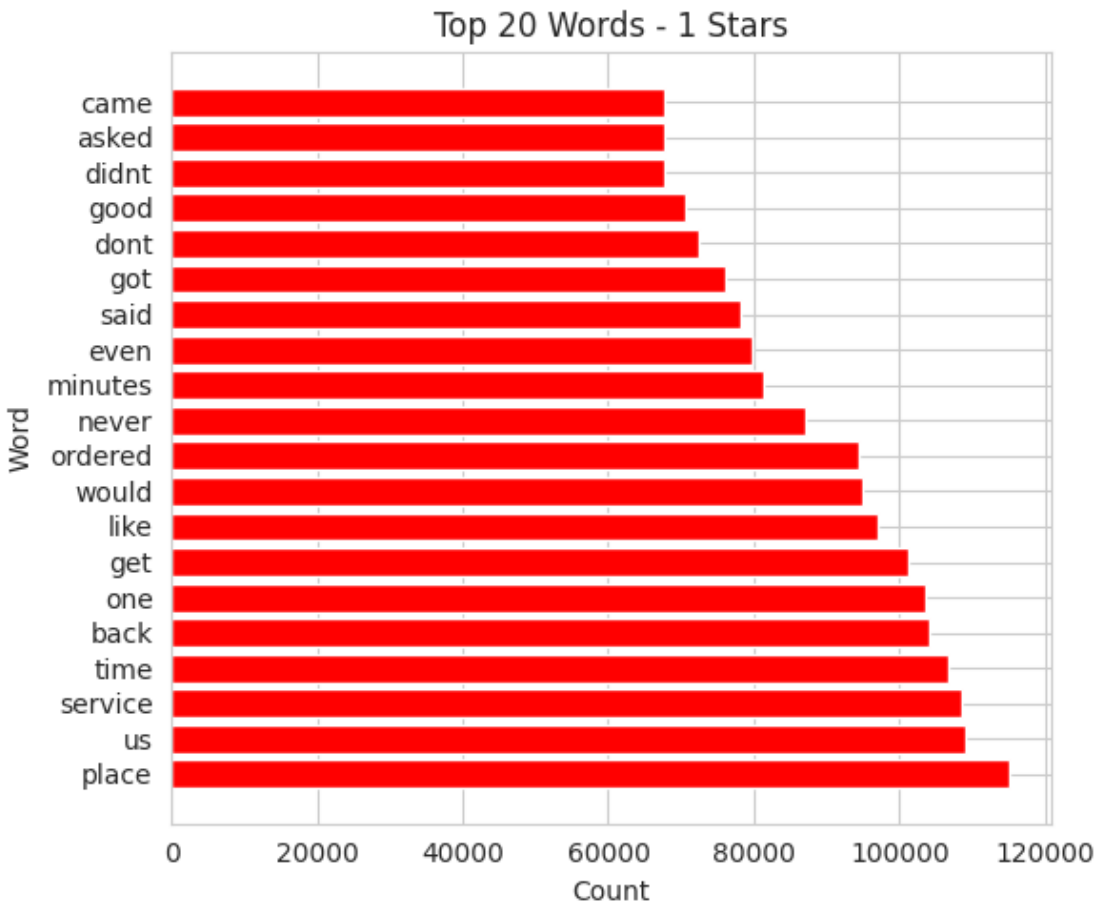

```
Pearson correlation between price tier and rating: 0.032
```

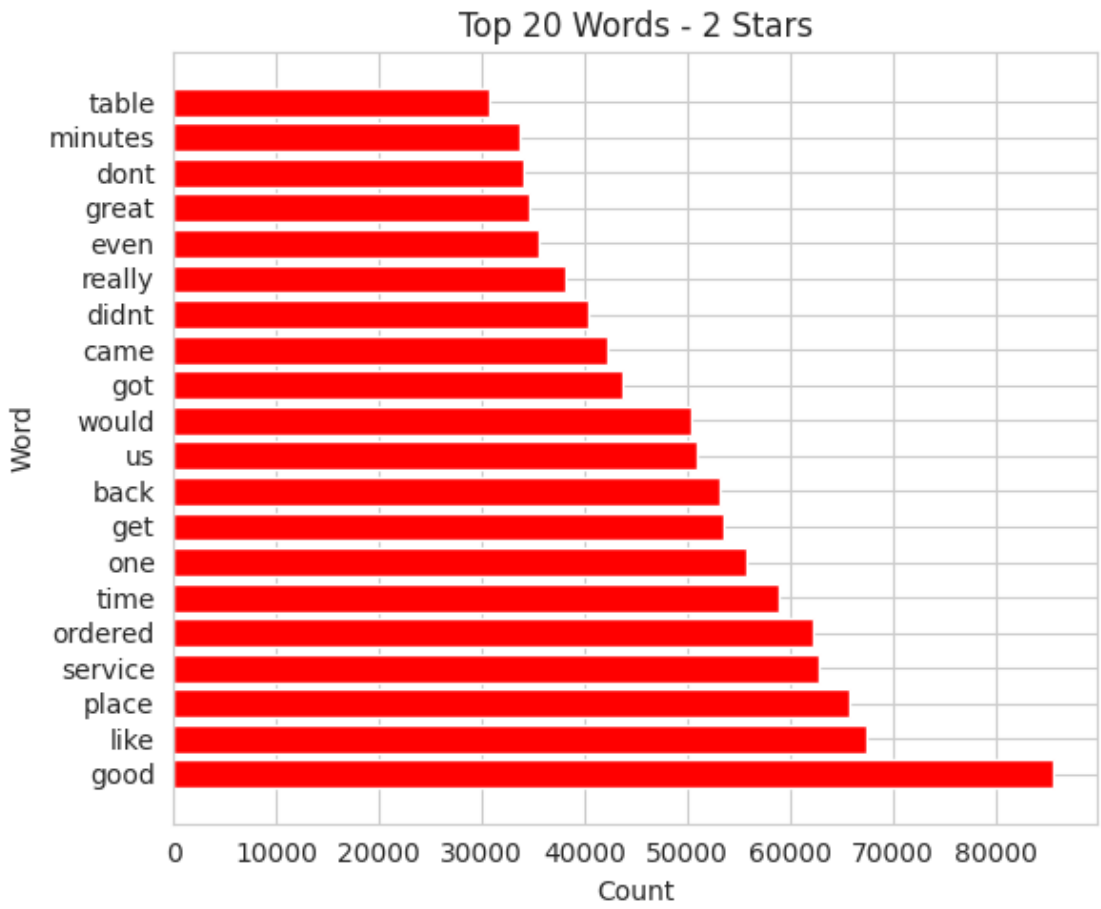
Since our dataset includes review text, we aimed to examine whether patterns exist in the most frequently used words across different star ratings. To do this, we performed word counts after removing stop words, allowing us to identify the top recurring terms within each of the five rating categories

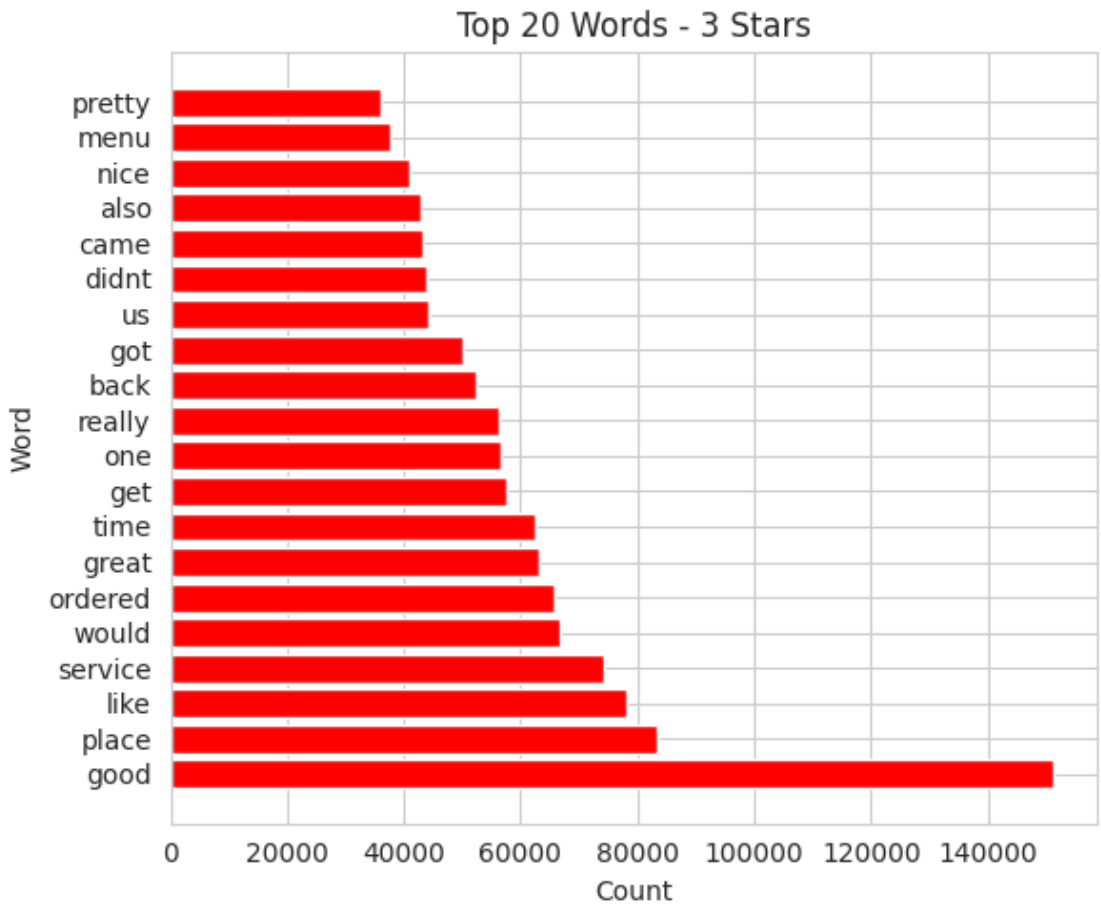
[25]:
```
# remove stopwords and create histograms of top 20 words by star rating

nltk.download('stopwords')

stop_words = set(stopwords.words('english'))
# Add custom words to remove
stop_words.update(['order', 'go', 'food', 'restaurant', 'chicken'])

for star in sorted(reviews_df['stars'].unique()):

    # Filter reviews_df for this star rating
    texts = reviews_df.loc[reviews_df['stars'] == star, 'text']
```

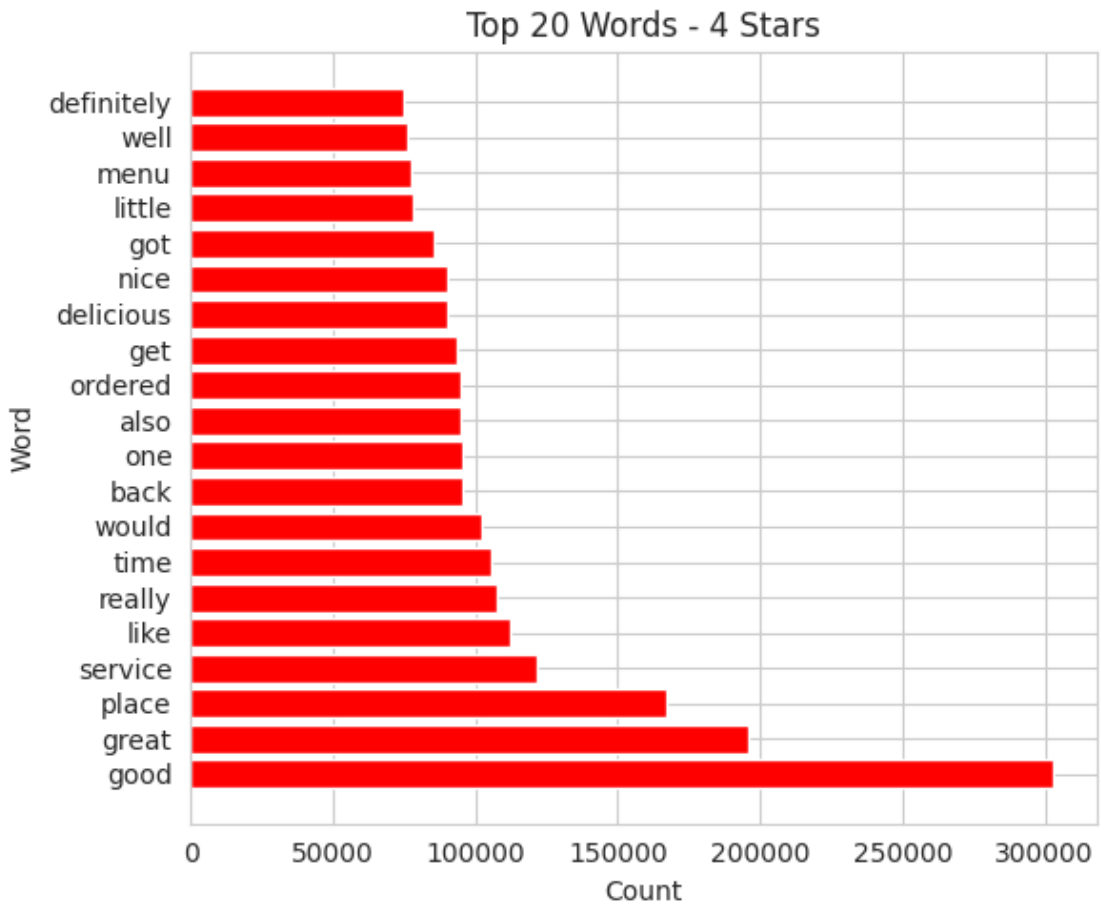
```
    # Clean + tokenize on the fly (no storage)
    words = []
    for text in texts:
        text = re.sub(r'[^a-z\s]', '', text.lower())
        tokens = [w for w in text.split() if w not in stop_words]
        words.extend(tokens)

    # Get top 20
    top20 = Counter(words).most_common(20)
    words_, counts_ = zip(*top20)

    #  Flip axes → horizontal bars
    plt.figure(figsize=(6,5))
    plt.barh(words_, counts_, color='red')

    plt.xlabel('Count')
    plt.ylabel('Word')
    plt.title(f'Top 20 Words - {int(star)} Stars')

    plt.tight_layout()
    plt.show()
```

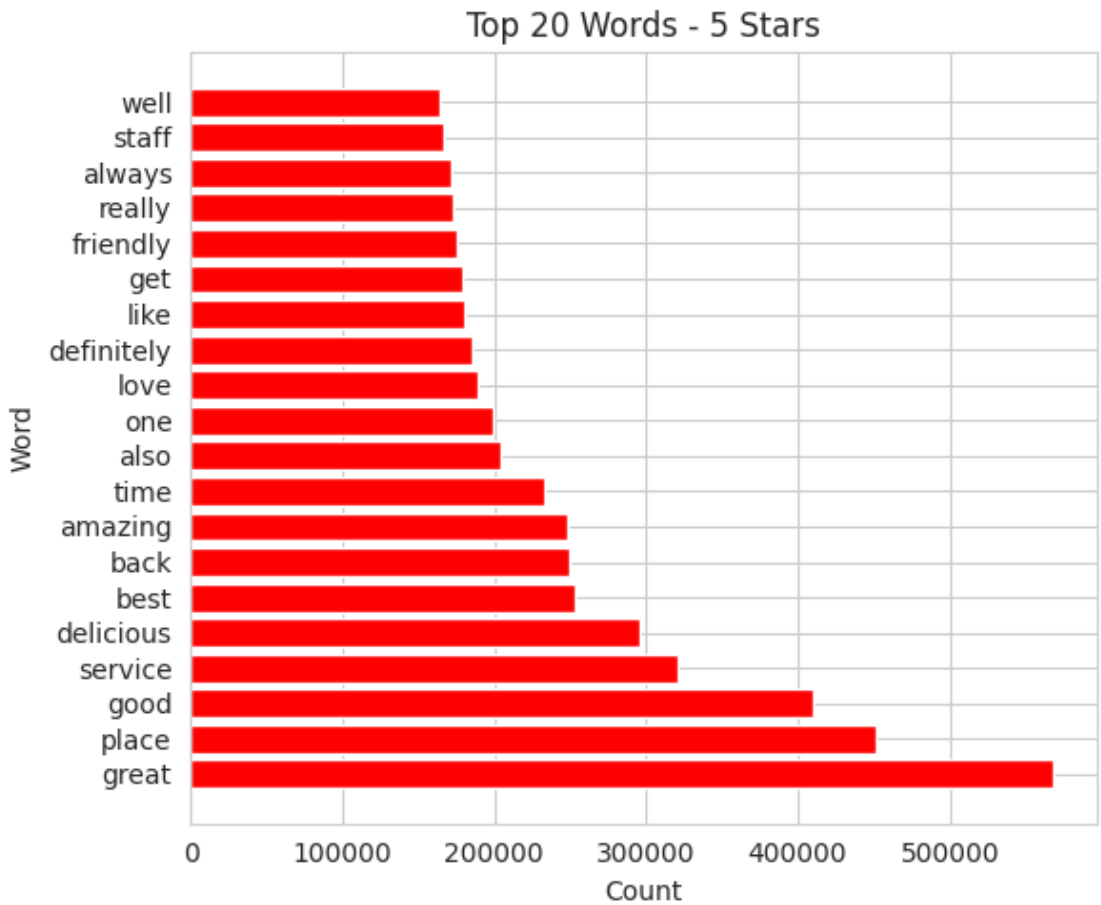
```
[nltk_data] Downloading package stopwords to /root/nltk_data…
[nltk_data]   Unzipping corpora/stopwords.zip.
```


Top 20 Words - 1 Stars
came
asked
didnt
good
dont
got
said
even
minutes
never
ordered
would
like
get
one
back
time
service
us
place
Word
0
20000
40000
60000
80000
100000
120000
Count


Top 20 Words - 2 Stars
table
minutes
dont
great
even
really
didnt
came
got
would
us
back
get
one
time
ordered
service
place
like
good
Word
0
10000
20000
30000
40000
50000
60000
70000
80000
Count


Top 20 Words - 3 Stars
Word
pretty
menu
nice
also
came
didnt
us
got
back
really
one
get
time
great
ordered
would
service
like
place
good
0
20000
40000
60000
80000
100000
120000
140000
Count


Top 20 Words - 4 Stars
definitely
well
menu
little
got
nice
delicious
get
ordered
also
one
back
would
time
really
like
service
place
great
good
Word
0
50000
100000
150000
200000
250000
300000
Count



Despite the boolean features containing mostly nulls, we still investigated potentential strong relationships with the average star ratings. While some of these boolean variables (i.e., features of the restaurant) did have a strong difference in star rating dependent on whether the variable was present, there were ultimately too many nulls to derive significance.

[26]:
```
# Creating & analyzing boolean features in Yelp data

# If you're in classic Jupyter, display exists; if not, this fallback prevents␣
↪errors
try:
    from IPython.display import display
except Exception:
    display = print

# ---- Safety checks ----
if "avg_rating" not in restaurant_df.columns:
    raise ValueError("restaurant_df must contain an 'avg_rating' column before␣
↪running this chunk.")
```

```python
# ---- 1) Detect boolean-like columns (robust to lists/dicts) ----
BOOL_TOKENS = {"true", "false", "yes", "no", "1", "0", "t", "f"}

bool_like = []
for col in restaurant_df.columns:
    if col == "avg_rating":
        continue
    try:
        uniq = restaurant_df[col].dropna().unique()
        uniq_str = {str(v).strip().lower() for v in uniq}
        if len(uniq_str) > 0 and uniq_str.issubset(BOOL_TOKENS):
            bool_like.append(col)
    except TypeError:
        # e.g., unhashable types like lists/dicts
        continue

print(f"Detected boolean-like columns: {bool_like}")

# ---- 2) Build bool_df with 0/1 encoding ----
# Convert typical boolean-ish values into 0/1, everything else -> NaN
def to01(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower()
    if s in {"true", "yes", "1", "t"}:
        return 1
    if s in {"false", "no", "0", "f"}:
        return 0
    return np.nan

bool_df = restaurant_df[["avg_rating"] + bool_like].copy()

for col in bool_like:
    bool_df[col] = bool_df[col].map(to01)

# ---- 3) Keep only boolean-like columns that actually vary (contain both 0 and␣
↪1) ----
valid_bool_like = []
for col in bool_like:
    vals = set(bool_df[col].dropna().unique())
    if vals == {0.0, 1.0} or vals == {0, 1}:
        valid_bool_like.append(col)

print(f"Valid boolean-like columns with variation: {valid_bool_like}")

# ---- 4) Effect metrics function (define BEFORE using it) ----
```

```python
def feature_effects(df, feature, target="avg_rating"):
    group_true = df.loc[df[feature] == 1, target].dropna()
    group_false = df.loc[df[feature] == 0, target].dropna()

    mean_true = group_true.mean() if len(group_true) else np.nan
    mean_false = group_false.mean() if len(group_false) else np.nan
    n_true = len(group_true)
    n_false = len(group_false)

    diff_abs = (
        abs(mean_true - mean_false)
        if np.isfinite(mean_true) and np.isfinite(mean_false)
        else np.nan
    )

    # Welch's t-test (handles unequal variances)
    if n_true >= 2 and n_false >= 2:
        _, p_value = ttest_ind(group_true, group_false, equal_var=False)
    else:
        p_value = np.nan

    # Cohen's d (optional, helpful)
    if n_true >= 2 and n_false >= 2:
        s1 = group_true.std(ddof=1)
        s0 = group_false.std(ddof=1)
        pooled = np.sqrt(((n_true - 1) * s1**2 + (n_false - 1) * s0**2) /␣
↪(n_true + n_false - 2))
        cohen_d = (mean_true - mean_false) / pooled if pooled and np.
↪isfinite(pooled) else np.nan
    else:
        cohen_d = np.nan

    return {
        "feature": feature,
        "mean_true": mean_true,
        "mean_false": mean_false,
        "diff_abs": diff_abs,
        "n_true": n_true,
        "n_false": n_false,
        "cohen_d": cohen_d,
        "p_value": p_value
    }

# ---- 5) Compute effects and show top 10 ----
effects_list = []
for col in valid_bool_like:
    try:
```

```
        effects_list.append(feature_effects(bool_df, col))
    except Exception as e:
        print(f"Error processing column {col}: {e}")

effects_df = pd.DataFrame(effects_list)

if effects_df.empty:
    print("No valid boolean-like columns found.")
else:
    top_diff = effects_df.sort_values("diff_abs", ascending=False).head(10)
    print("Top 10 features by absolute rating difference:")
    display(top_diff[["feature", "mean_true", "mean_false", "diff_abs",␣
 ↪"n_true", "n_false", "cohen_d", "p_value"]])

    # ---- 6) Value counts for the top features (replaces your Cell 5) ----
    value_counts_df = pd.DataFrame({
        feat: bool_df[feat].value_counts(dropna=False)
        for feat in top_diff["feature"]
    }).T.fillna(0).astype(int)

    # Rename columns if present
    rename_map = {}
    if 0 in value_counts_df.columns: rename_map[0] = "False (0)"
    if 1 in value_counts_df.columns: rename_map[1] = "True (1)"
    if np.nan in value_counts_df.columns: rename_map[np.nan] = "NaN"
    value_counts_df = value_counts_df.rename(columns=rename_map)

    print("\nValue counts (sample sizes) for top features:")
    display(value_counts_df)
```

```
Detected boolean-like columns: ['BusinessAcceptsCreditCards',
'ByAppointmentOnly', 'GoodForKids', 'CoatCheck', 'DogsAllowed',
'RestaurantsGoodForGroups', 'WheelchairAccessible', 'HappyHour',
'BusinessAcceptsBitcoin', 'GoodForDancing', 'BYOB', 'Corkage']
Valid boolean-like columns with variation: ['BusinessAcceptsCreditCards',
'ByAppointmentOnly', 'GoodForKids', 'CoatCheck', 'DogsAllowed',
'RestaurantsGoodForGroups', 'WheelchairAccessible', 'HappyHour',
'BusinessAcceptsBitcoin', 'GoodForDancing', 'BYOB', 'Corkage']
Top 10 features by absolute rating difference:

                       feature  mean_true  mean_false  diff_abs  n_true  \
1            ByAppointmentOnly   4.037037    3.287536  0.749501       1
8       BusinessAcceptsBitcoin   4.206439    3.497258  0.709182       2
3                    CoatCheck   4.038537    3.398052  0.640485       2
10                        BYOB   3.562497    3.236032  0.326466      18
6         WheelchairAccessible   3.510753    3.234662  0.276090     212
11                     Corkage   3.322753    3.564945  0.242193      16
0   BusinessAcceptsCreditCards   3.494977    3.264897  0.230081     789
```

```
9               GoodForDancing   3.551998    3.400422  0.151576       9
7                    HappyHour   3.496973    3.394696  0.102278     163
2                  GoodForKids   3.497380    3.560163  0.062783     632

    n_false   cohen_d   p_value
1        68       NaN       NaN
8       111  0.705891  0.176896
3        74  0.643155  0.222197
10       50  0.311001  0.225792
6        20  0.287483  0.248980
11       48 -0.240528  0.401948
0        23  0.232529  0.275877
9        62  0.146858  0.608755
7        77  0.099551  0.469449
2       113 -0.063289  0.510636


Value counts (sample sizes) for top features:

                            False (0)  True (1)    NaN
ByAppointmentOnly                  68         1  42205
BusinessAcceptsBitcoin            111         2  42161
CoatCheck                          74         2  42198
BYOB                               50        18  42206
WheelchairAccessible               20       212  42042
Corkage                            48        16  42210
BusinessAcceptsCreditCards         23       789  41462
GoodForDancing                     62         9  42203
HappyHour                          77       163  42034
GoodForKids                       113       632  41529
```

1.2.1 Subcategory Analysis

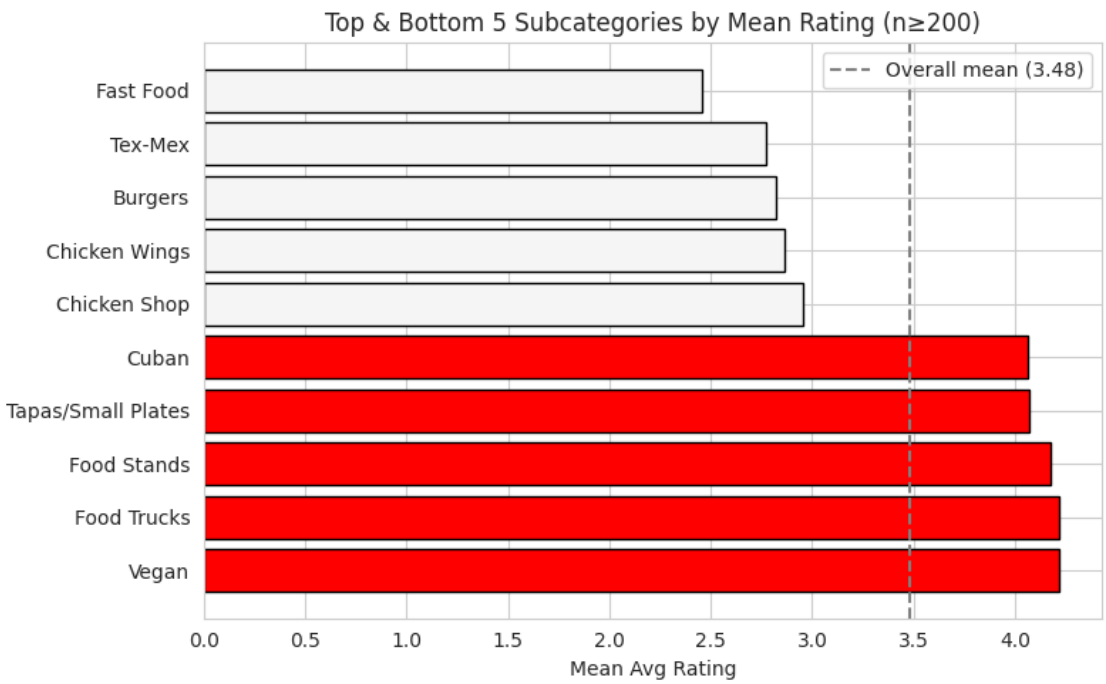
Since many restaurants were associated with a secondary business category (i.e., restaurant type), we sought to examine how these categories relate to star ratings. To explore this relationship, we plotted the rating distributions for the five secondary business types with the highest average ratings and the five with the lowest average ratings.

[27]:
```python
from scipy.stats import f_oneway

# Explode secondary categories into one row each
exploded = restaurant_df[['secondary_category', 'avg_rating']].
 ↪explode('secondary_category')
exploded['secondary_category'] = exploded['secondary_category'].str.strip()

# Aggregate: mean rating + count per subcategory
cat_stats = (
    exploded.groupby('secondary_category')['avg_rating']
    .agg(mean_rating='mean', count='count')
```

```
    .reset_index()
)

# Filter to meaningful sample size
MIN_COUNT = 200
cat_stats = cat_stats[cat_stats['count'] >= MIN_COUNT].
 ↪sort_values('mean_rating', ascending=False)

# Summary stats
overall_mean = restaurant_df['avg_rating'].mean()
print(f"Overall mean rating : {overall_mean:.3f}")
print(f"Subcategories analysed (n>={MIN_COUNT}): {len(cat_stats)}\n")
print("Top 5 highest-rated subcategories:")
print(cat_stats.head(5).to_string(index=False))
print("\nTop 5 lowest-rated subcategories:")
print(cat_stats.tail(5).to_string(index=False))

# ANOVA: do subcategories differ significantly?
groups = [
    group['avg_rating'].values
    for _, group in exploded.groupby('secondary_category')
    if len(group) >= MIN_COUNT
]
f_stat, p_value = f_oneway(*groups)
print(f"\nOne-way ANOVA  →  F={f_stat:.2f},  p={p_value:.2e}")
print("Subcategories differ significantly in rating." if p_value < 0.05 else␣
 ↪"No significant difference found.")

# Plot
top_n = 10
plot_df = pd.concat([cat_stats.head(top_n // 2), cat_stats.tail(top_n // 2)])
plt.figure(figsize=(8, 5))
colors = ['red' if r >= overall_mean else 'whitesmoke' for r in␣
 ↪plot_df['mean_rating']]
plt.barh(plot_df['secondary_category'], plot_df['mean_rating'], color=colors,␣
 ↪edgecolor='black')
plt.axvline(overall_mean, linestyle='--', color='gray', label=f'Overall mean␣
 ↪({overall_mean:.2f})')
plt.xlabel('Mean Avg Rating')
plt.title(f'Top & Bottom {top_n//2} Subcategories by Mean Rating␣
 ↪(n {MIN_COUNT})')
plt.legend()
plt.tight_layout()
plt.show()
```

```
Overall mean rating : 3.480
Subcategories analysed (n>=200): 94
```

```
Top 5 highest-rated subcategories:
secondary_category  mean_rating  count
             Vegan     4.217209    745
       Food Trucks     4.217038    863
       Food Stands     4.176445    306
Tapas/Small Plates     4.073385    364
             Cuban     4.066108    218

Top 5 lowest-rated subcategories:
secondary_category  mean_rating  count
      Chicken Shop     2.956960    795
     Chicken Wings     2.866126   2722
           Burgers     2.819426   5030
           Tex-Mex     2.775535    824
         Fast Food     2.457518   6102

One-way ANOVA  →  F=237.93,  p=0.00e+00
Subcategories differ significantly in rating.
```



1.2.2 Extract and analyze correlation of reviews' sentiment polarity, subjectivity, and length

We used TextBlob to score and average sentiment and subjectivity for each business–and also measured review length in order to explore potential correlation with average star rating.

[28]:
```
from tqdm.auto import tqdm
tqdm.pandas()

# Extract all three features at once
def get_text_features(text):
    blob = TextBlob(str(text))
    return blob.sentiment.polarity, blob.sentiment.subjectivity, len(text)

tqdm_text.pandas(desc="Extracting text features")
reviews_df[["sentiment_polarity", "sentiment_subjectivity", "review_length"]] =␣
↪(
    reviews_df["text"]
    .progress_apply(lambda x: pd.Series(get_text_features(x)))
)

# Aggregate all three for EDA correlation analysis
eda_cols = ["avg_sentiment_polarity", "avg_sentiment_subjectivity",␣
↪"avg_review_length"]
eda_agg = reviews_df.groupby("business_id")[["sentiment_polarity",␣
↪"sentiment_subjectivity", "review_length"]].mean()
eda_agg.columns = eda_cols

eda_temp = restaurant_df.merge(eda_agg, on="business_id", how="left").fillna(0)
for col in eda_cols:
    r = eda_temp[col].corr(eda_temp["avg_rating"])
    print(f"Correlation between '{col}' and 'avg_rating': {r:.4f}")
```

```
Extracting text features: 100%|      | 2007019/2007019 [21:56<00:00,
1524.53it/s]
```

```
Correlation between 'avg_sentiment_polarity' and 'avg_rating': 0.8449
Correlation between 'avg_sentiment_subjectivity' and 'avg_rating': 0.3075
Correlation between 'avg_review_length' and 'avg_rating': -0.0720
```

2 Feature creation

Given the large amount of words in reviews, we wanted to see if we could create a feature using words commonly used in 5-star reviews. We computed TF-IDF to find the most importnat words in the top 500 words used, we then correlated them to 5-star reveiws and picked the 5 top words with the highest correlation.

Using the top 5 words, we then created a dichotomous flagging feature, that flags a review if it has that word. Using this dichotomous feature, we computed another correlation against 5-star reveiws, the correlation value is 0.36, a moderate but low correlation, but still higher than the words on their own.

We concluded that a more powerful feature should be built using sentiment analysis that uses a group of words, rather than single words alone.

[29]:
```
# Binary target: 1 if 5-star else 0
y = (reviews_df["stars"] == 5).astype(np.float32).values
y_mean = y.mean()
y_centered = y - y_mean
y_std = y_centered.std()

vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)
X = vectorizer.fit_transform(reviews_df["text"].fillna(""))

# Center X columns: we do it implicitly via means
# cov(X_j, y) = mean(X_j * y) - mean(X_j) * mean(y)
n = X.shape[0]

x_mean = np.asarray(X.mean(axis=0)).ravel()                    # (p,)
xy_mean = (X.T @ y) / n                                        # (p,)␣
 ↪sparse->dense vector
xy_mean = np.asarray(xy_mean).ravel()

cov = xy_mean - x_mean * y_mean

# std(X_j)
# E[X^2] - (E[X])^2
x2_mean = np.asarray(X.multiply(X).mean(axis=0)).ravel()
x_std = np.sqrt(np.maximum(x2_mean - x_mean**2, 1e-12))

corr = cov / (x_std * max(y_std, 1e-12))

feature_names = vectorizer.get_feature_names_out()
corr_s = pd.Series(corr, index=feature_names).sort_values(ascending=False)

print("Top 5 words most correlated with 5-star reviews:")
print(corr_s.head(5))
```

```
Top 5 words most correlated with 5-star reviews:
amazing      0.225219
great        0.209020
delicious    0.207256
best         0.198333
highly       0.138648
dtype: float64
```

We were curious to see if a dichotomous feature that flags reviews for having positive words would show a correlation with avg_rating. Using the already loaded 'reviews_df' dataset.

[30]:
```
# Define keywords for positive words
positive_keywords = ['amazing', 'great', 'delicious','best', 'highly']
```

```python
# Create binary flag at review level (vectorized, more efficient than custom␣
↪function)
reviews_df['has_positive_keywords'] = (
    reviews_df['text']
    .str.lower()
    .apply(lambda x: int(any(word in x for word in positive_keywords)))
)

# Aggregate to business level:
# 1 if at least one review contains positive keywords, else 0
business_positive_keywords = (
    reviews_df
    .groupby('business_id')['has_positive_keywords']
    .max()
    .reset_index()
    .rename(columns={'has_positive_keywords': 'review_has_pos_words'})
)

# Merge into restaurant_df
restaurant_df = restaurant_df.merge(business_positive_keywords,␣
↪on='business_id', how='left')

# Fill NaN (businesses without matching reviews)
restaurant_df['review_has_pos_words'] = (
    restaurant_df['review_has_pos_words']
    .fillna(0)
    .astype(int)
)

print("Created 'review_has_pos_words' feature.")
display(restaurant_df[['business_id', 'review_has_pos_words']].head())
```

```
Created 'review_has_pos_words' feature.
```

	business_id	review_has_pos_words
0	MTSW4McQd7CbVtyjqoe9mw	1
1	CF33F8-E6oudUQ46HnavjQ	0
2	bBDDEgkFA1Otx9Lfe7BZUQ	0
3	eEOYSgkmpB90uNA7lDOMRA	1
4	il_Ro8jwPlHresjw9EGmBg	1

[31]:
```python
correlation = restaurant_df['review_has_pos_words'].
↪corr(restaurant_df['avg_rating'])
print(f"Correlation between 'review_has_pos_words' and 'avg_rating':␣
↪{correlation:.4f}")
```

```
Correlation between 'review_has_pos_words' and 'avg_rating': 0.3607
```

Given the strong correlation between average review polarity and subjectivity score with average

rating, we opted to create two new features.

[32]:
```
# Add sentiment features to restaurant_df (review_length excluded - correlation:
↪ -0.07)
sentiment_cols = ["avg_sentiment_polarity", "avg_sentiment_subjectivity"]
restaurant_df = restaurant_df.drop(columns=[c for c in sentiment_cols if c in␣
↪restaurant_df.columns])

agg = reviews_df.groupby("business_id")[["sentiment_polarity",␣
↪"sentiment_subjectivity"]].mean()
agg.columns = sentiment_cols
restaurant_df = restaurant_df.merge(agg, on="business_id", how="left").fillna(0)
```

[33]:
```
# Verify the new features
print(restaurant_df[["business_id", "avg_sentiment_polarity",␣
↪"avg_sentiment_subjectivity"]].head())
```

```
              business_id  avg_sentiment_polarity  avg_sentiment_subjectivity
0  MTSW4McQd7CbVtyjqoe9mw                0.341137                    0.544293
1  CF33F8-E6oudUQ46HnavjQ               -0.263393                    0.526625
2  bBDDEgkFA1Otx9Lfe7BZUQ                0.075430                    0.508649
3  eEOYSgkmpB90uNA7lDOMRA                0.196803                    0.597329
4  il_Ro8jwPlHresjw9EGmBg                0.002938                    0.521880
```

2.0.1 Mean subcategory rating score feature

Considering there were substantial differences between the restaurants' subcategories and average star rating, we created a new continous variable feature reflecting the average rating by subcategory.

[34]:
```
# Lookup from EDA secondary category analysis
cat_mean_lookup = cat_stats.set_index('secondary_category')['mean_rating'].
↪to_dict()

# Average the mapped ratings across all subcategories
restaurant_df['subcategory_rating_score'] = restaurant_df['secondary_category'].
↪apply(
    lambda cats: np.mean([cat_mean_lookup[c] for c in cats if c in␣
↪cat_mean_lookup])
    if isinstance(cats, list) and any(c in cat_mean_lookup for c in cats) else␣
↪np.nan
)
```

[35]:
```
# Verify the new feature
print(restaurant_df[["business_id", "subcategory_rating_score"]].head())
```

```
              business_id  subcategory_rating_score
0  MTSW4McQd7CbVtyjqoe9mw                  3.683847
1  CF33F8-E6oudUQ46HnavjQ                  3.127553
2  bBDDEgkFA1Otx9Lfe7BZUQ                  3.050376
```

```
3  eEOYSgkmpB90uNA7lDOMRA                    3.920059
4  il_Ro8jwPlHresjw9EGmBg                    3.438537
```

2.0.2 Mean city rating score feature

[36]:
```
# Mean-encode city - cities with <30 businesses get global mean as fallback
city_counts = restaurant_df.groupby('city')['avg_rating'].count()
city_mean   = restaurant_df.groupby('city')['avg_rating'].mean()

valid_cities = city_counts[city_counts >= 30].index
global_mean  = restaurant_df['avg_rating'].mean()

restaurant_df['city_mean_rating'] = (
    restaurant_df['city']
    .map(city_mean[city_mean.index.isin(valid_cities)])
    .fillna(global_mean)
)
```

[37]:
```
# Verify the new feature
print(restaurant_df[["business_id", "city_mean_rating"]].head())
```

```
              business_id  city_mean_rating
0  MTSW4McQd7CbVtyjqoe9mw          3.552488
1  CF33F8-E6oudUQ46HnavjQ          3.480349
2  bBDDEgkFA1Otx9Lfe7BZUQ          3.541494
3  eEOYSgkmpB90uNA7lDOMRA          3.480349
4  il_Ro8jwPlHresjw9EGmBg          3.465031
```

3 Modeling

3.1 2 Text Analysis Models for Review Rating Prediction

3.1.1 Support Vector Machine (SVM)

[38]:
```
# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

# 1) Target + features
print("Step 1/6: Preparing features...")

binary_df = reviews_df[reviews_df["stars"] != 3].copy()

X_text = binary_df["text"].fillna("")
y = binary_df["stars"].apply(lambda x: 1 if x >= 4 else 0)

# 2) Train/test split (stratified)
print("Step 2/6: Splitting data...")
```

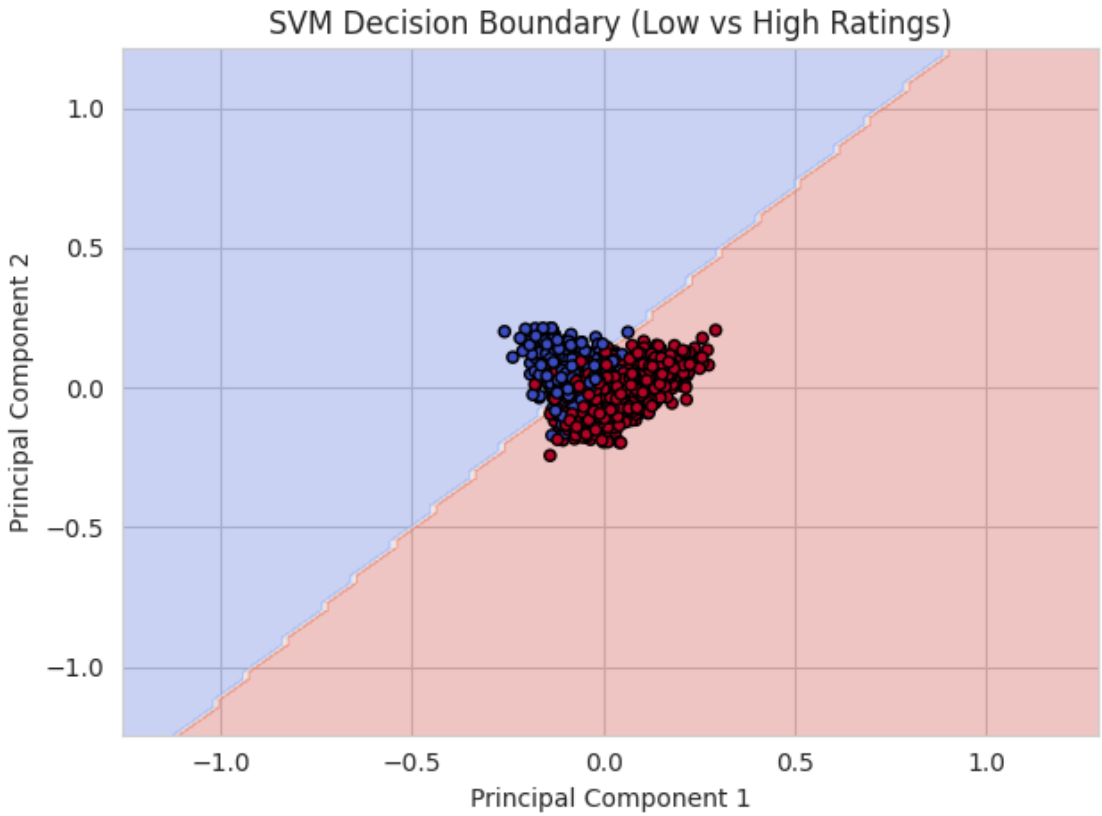
```python
X_train, X_test, y_train, y_test = train_test_split(
    X_text,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 3) TF-IDF (fit ONLY on training)
print("Step 3/6: Vectorizing text (TF-IDF)...")
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=9000,
    ngram_range=(1, 2)
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# 4) Cross-validation
print("Step 4/6: Running cross-validation...")
svm_model = LinearSVC(class_weight="balanced", random_state=42)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(
    svm_model,
    X_train_vec,
    y_train,
    cv=cv,
    n_jobs=1
)

# 5) Final fit + evaluation
print("Step 5/6: Final fit and evaluation...")
svm_model.fit(X_train_vec, y_train)
y_pred = svm_model.predict(X_test_vec)

print(f"\nCV Accuracy : {round(cv_scores.mean(), 4)}")
print(f"CV Std Dev  : {round(cv_scores.std(), 4)}")

print(f"\nTest Accuracy: {round(accuracy_score(y_test, y_pred), 4)}")
print(
    "\nClassification Report:\n",
    classification_report(
        y_test,
        y_pred,
        zero_division=0,
```

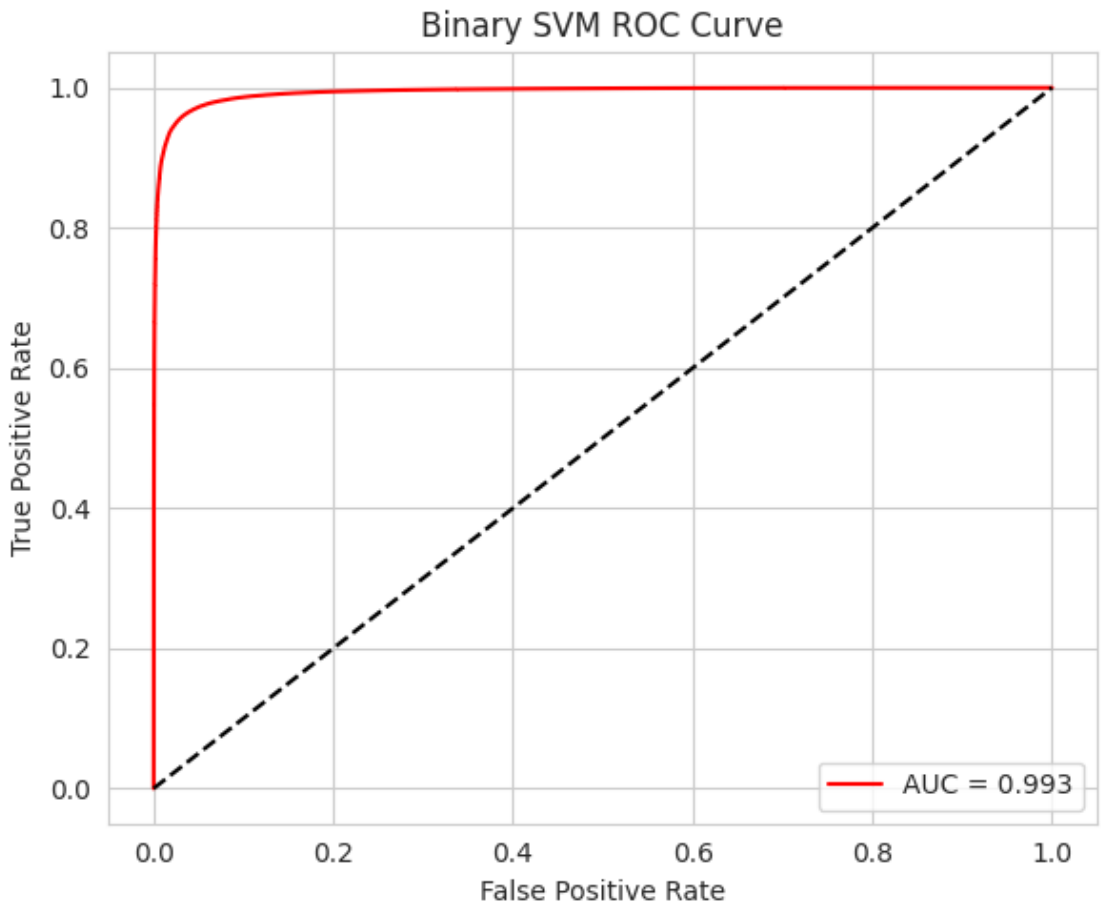
```
        target_names=["Low Rating (1-2)", "High Rating (4-5)"]
    )
)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# 6) Visualizations
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.metrics import roc_curve, auc

print("Step 6/6: Plotting decision boundary and ROC curve...")

# Sample for visualization
sample_n = 3000
X_sample = X_test_vec[:sample_n].toarray()
y_sample = y_test.iloc[:sample_n].values

# PCA projection
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_sample)

# Train small SVM for visualization only
svm_vis = SVC(kernel="linear", random_state=42)
svm_vis.fit(X_pca, y_sample)

plt.figure(figsize=(7,5))
DecisionBoundaryDisplay.from_estimator(
    svm_vis,
    X_pca,
    response_method="predict",
    cmap="coolwarm",
    alpha=0.3
)

plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=y_sample,
    cmap="coolwarm",
    edgecolor="k",
    s=20
)

plt.title("SVM Decision Boundary (Low vs High Ratings)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

```
plt.tight_layout()
plt.show()

# ROC curve
y_score = svm_model.decision_function(X_test_vec)
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}" , color = 'red')
plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Binary SVM ROC Curve")
plt.legend()
plt.tight_layout()
plt.show()

# Time & RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
print(f"\nTotal time : {time.time() - start_time:.1f}s")
print(f"RAM delta  : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
Step 1/6: Preparing features…
Step 2/6: Splitting data…
Step 3/6: Vectorizing text (TF-IDF)…
Step 4/6: Running cross-validation…
Step 5/6: Final fit and evaluation…

CV Accuracy : 0.9632
CV Std Dev  : 0.0003

Test Accuracy: 0.9629

Classification Report:
                    precision    recall  f1-score   support

 Low Rating (1-2)       0.89      0.96      0.93     86779
High Rating (4-5)       0.99      0.96      0.98    277435

         accuracy                           0.96    364214
        macro avg       0.94      0.96      0.95    364214
     weighted avg       0.96      0.96      0.96    364214


Confusion Matrix:
 [[ 83667   3112]
```

```
 [ 10395 267040]]
Step 6/6: Plotting decision boundary and ROC curve…

<Figure size 700x500 with 0 Axes>
```






```
Total time : 425.2s
RAM delta  : 1169.5 MB  (total: 7826.9 MB)
```

3.1.2 Long Short-Term Memory (LSTM) Recurrent Neural Network (RNN)

[39]:
```
# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

# 1) Features and target
print("Step 1/6: Preparing features and target...")
X_text = reviews_df["text"].fillna("").astype(str)
y = reviews_df["stars"].astype(int) - 1   # convert 1-5 to 0-4

# 2) Train/test split
print("Step 2/6: Splitting data into train and test sets...")
X_train, X_test, y_train, y_test = train_test_split(
```

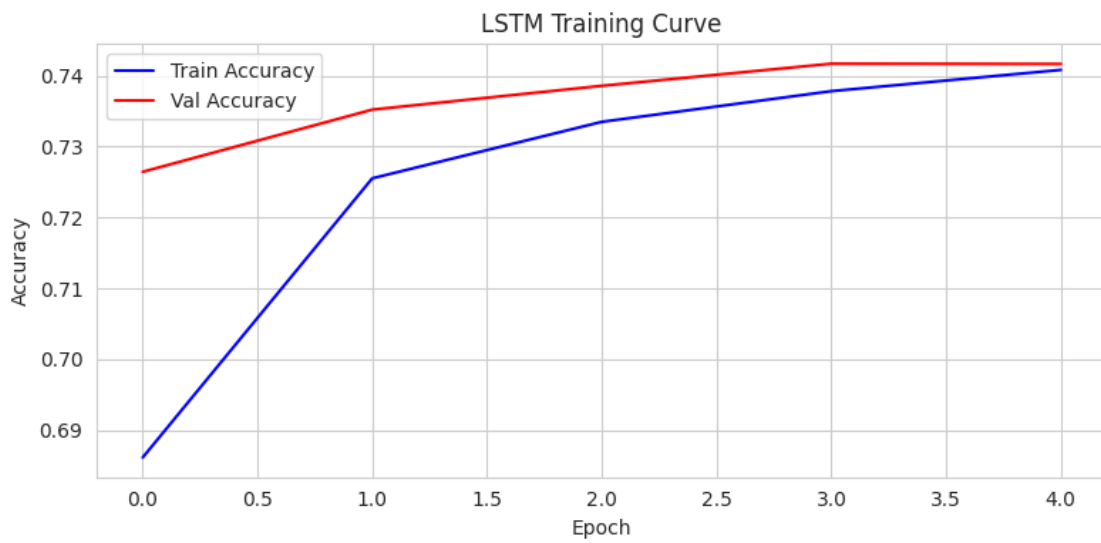
```
    X_text,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 3) Tokenize and pad
print("Step 3/6: Tokenizing and padding sequences...")
max_words = 5000
max_seq_length = 100

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(
    X_train_seq,
    maxlen=max_seq_length,
    padding="post",
    truncating="post"
)

X_test_pad = pad_sequences(
    X_test_seq,
    maxlen=max_seq_length,
    padding="post",
    truncating="post"
)

# 4) One-hot encode target
print("Step 4/6: One-hot encoding target...")
y_train_cat = to_categorical(y_train, num_classes=5)
y_test_cat = to_categorical(y_test, num_classes=5)

# 5) Build and compile model
print("Step 5/6: Building and compiling the LSTM model...")
model = Sequential([
    Embedding(input_dim=max_words, output_dim=64),
    LSTM(64, dropout=0.2),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(5, activation="softmax")
])
```

```python
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=1,
    restore_best_weights=True
)

print("Training rows:", len(X_train_pad))
print("Batch size:", 64)
print("Expected steps per epoch:", int(np.ceil(len(X_train_pad) / 64)))

# Train
history = model.fit(
    X_train_pad,
    y_train_cat,
    validation_split=0.1,
    epochs=10,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)

# 6) Evaluate
print("Step 6/6: Evaluating...")
y_pred = model.predict(X_test_pad, verbose=0).argmax(axis=1)

# Remap back to 1-5 for readable output
y_test_display = y_test + 1
y_pred_display = y_pred + 1

print(f"\nTest Accuracy: {round(accuracy_score(y_test, y_pred), 4)}")
print("\nClassification Report:\n", classification_report(y_test_display,␣
 ↪y_pred_display, zero_division=0))
print("\nConfusion Matrix:\n", confusion_matrix(y_test_display, y_pred_display))

# Training curve
plt.figure(figsize=(8, 4))
plt.plot(history.history["accuracy"], label="Train Accuracy" , color = 'blue')
plt.plot(history.history["val_accuracy"], label="Val Accuracy" , color = 'red')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("LSTM Training Curve")
```

```
plt.legend()
plt.tight_layout()
plt.show()

# Time and RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
print(f"\nTotal time : {time.time() - start_time:.1f}s")
print(f"RAM delta  : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
Step 1/6: Preparing features and target…
Step 2/6: Splitting data into train and test sets…
Step 3/6: Tokenizing and padding sequences…
Step 4/6: One-hot encoding target…
Step 5/6: Building and compiling the LSTM model…
Training rows: 1605615
Batch size: 64
Expected steps per epoch: 25088
Epoch 1/10
22579/22579              1125s
50ms/step - accuracy: 0.6861 - loss: 0.7752 - val_accuracy: 0.7264 - val_loss:
0.6596
Epoch 2/10
22579/22579              1123s
50ms/step - accuracy: 0.7255 - loss: 0.6682 - val_accuracy: 0.7352 - val_loss:
0.6375
Epoch 3/10
22579/22579              1110s
49ms/step - accuracy: 0.7335 - loss: 0.6479 - val_accuracy: 0.7386 - val_loss:
0.6298
Epoch 4/10
22579/22579              1167s
49ms/step - accuracy: 0.7378 - loss: 0.6363 - val_accuracy: 0.7417 - val_loss:
0.6228
Epoch 5/10
22579/22579              1194s
51ms/step - accuracy: 0.7408 - loss: 0.6290 - val_accuracy: 0.7416 - val_loss:
0.6232
Step 6/6: Evaluating…

Test Accuracy: 0.7422

Classification Report:
               precision    recall  f1-score   support

           1       0.74      0.85      0.79     56140
           2       0.50      0.37      0.42     30639
           3       0.55      0.48      0.51     37190
```

```
           4       0.58      0.46      0.52     73774
           5       0.84      0.92      0.88    203661

    accuracy                           0.74    401404
   macro avg       0.64      0.61      0.62    401404
weighted avg       0.72      0.74      0.73    401404


Confusion Matrix:
 [[ 47522   5663   1633    380    942]
 [ 11090  11269   6551   1026    703]
 [  3345   5296  17804   8382   2363]
 [   978    430   5435  34130  32801]
 [  1099     93    715  14573 187181]]
```




```
Total time : 6044.8s
RAM delta  : 2645.0 MB  (total: 10471.9 MB)
```

3.2 4 Predictive Models for Restaurant Rating Averages

3.2.1 Random Forest Model: Boolean Features

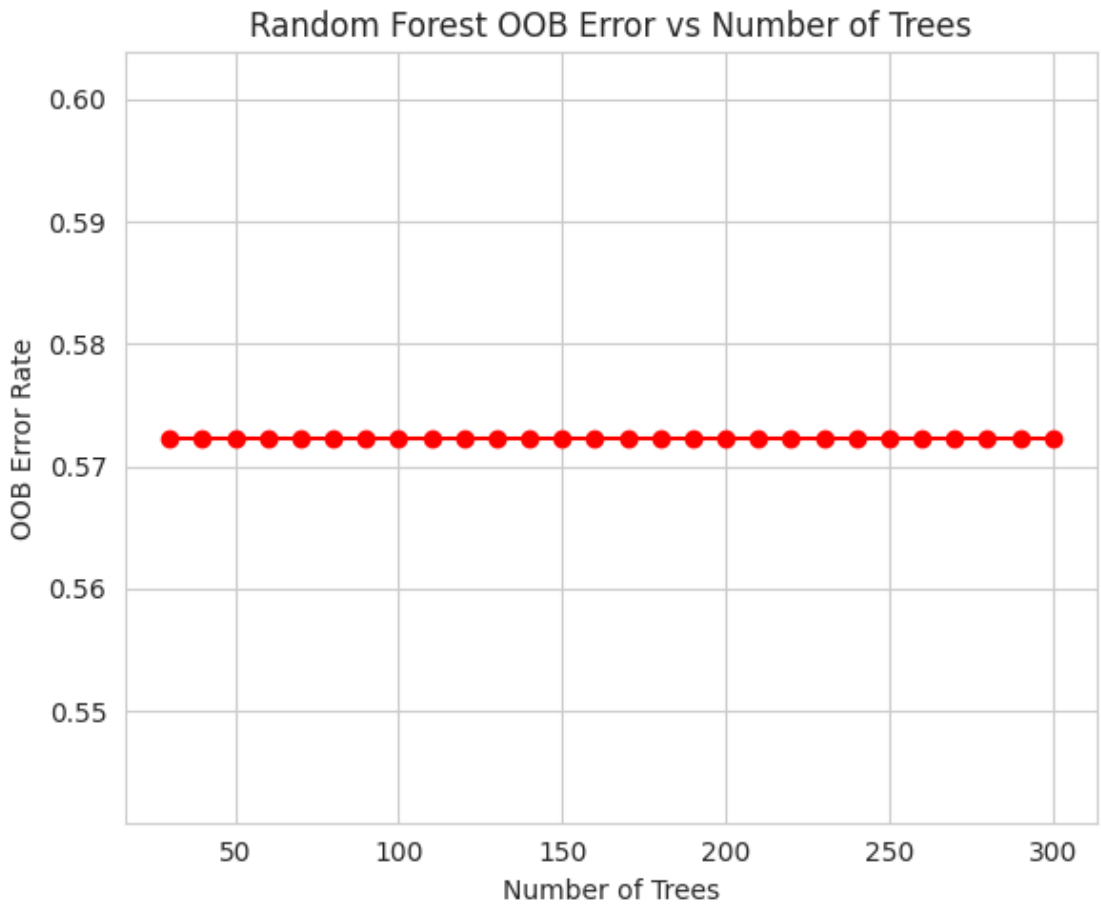
[40]:
```python
# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

# Select boolean columns
bool_cols = [col for col in restaurant_df.columns
             if restaurant_df[col].dropna().isin([0,1]).all()]
```

```python
print(f"Boolean columns found: {len(bool_cols)}")

# Feature matrix and NEW binary target
X = restaurant_df[bool_cols]

y = (restaurant_df["avg_rating"] >= 4).astype(int)
# 1 = High rating restaurant
# 0 = Low rating restaurant

# Train/test split (now stratified on binary target)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=7,
    min_samples_leaf=5,
    class_weight="balanced",
    n_jobs=-1,
    random_state=42,
    bootstrap=True,
    oob_score=True
)

# Cross-validation
print("Running 5-fold cross-validation...")
cv_scores = cross_val_score(
    rf_model,
    X_train,
    y_train,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    n_jobs=1
)
print("CV Accuracy:", round(cv_scores.mean(), 4))
print("CV Std Dev :", round(cv_scores.std(), 4))

# OOB error curve
import warnings
warnings.filterwarnings("ignore")

print("\nCalculating OOB error curve...")
oob_rf = RandomForestClassifier(
    n_estimators=30,
    max_depth=7,
    min_samples_leaf=5,
```

```
    class_weight="balanced",
    n_jobs=-1,
    random_state=42,
    bootstrap=True,
    oob_score=True,
    warm_start=True
)

oob_errors = []
tree_range = range(30, 301, 10)

for n_trees in tree_range:
    oob_rf.set_params(n_estimators=n_trees)
    oob_rf.fit(X_train, y_train)
    oob_errors.append(1 - oob_rf.oob_score_)

plt.figure(figsize=(6, 5))
plt.plot(tree_range, oob_errors, marker="o", color="red")
plt.xlabel("Number of Trees")
plt.ylabel("OOB Error Rate")
plt.title("Random Forest OOB Error vs Number of Trees")
plt.tight_layout()
plt.show()

# Train final model
print("\nTraining final model...")
rf_model.fit(X_train, y_train)
print("Done.")

# Predictions
y_pred = rf_model.predict(X_test)

print("\nTest Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print(
    "\nClassification Report:\n",
    classification_report(
        y_test,
        y_pred,
        zero_division=0,
        target_names=["Low Rating (<4)", "High Rating ( 4)"]
    )
)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Time & RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
print(f"\nTime elapsed : {time.time() - start_time:.1f}s")
```

```
print(f"RAM delta     : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
Boolean columns found: 7
Running 5-fold cross-validation…
CV Accuracy: 0.4277
CV Std Dev : 0.0045

Calculating OOB error curve…
```




```
Training final model…
Done.

Test Accuracy: 0.4308

Classification Report:
                   precision    recall  f1-score   support
```

```
 Low Rating (<4)       0.88      0.10      0.18      5263
High Rating ( 4)       0.40      0.98      0.56      3192

        accuracy                           0.43      8455
       macro avg       0.64      0.54      0.37      8455
    weighted avg       0.70      0.43      0.32      8455


Confusion Matrix:
 [[ 521 4742]
 [  71 3121]]

Time elapsed : 20.9s
RAM delta    : -472.5 MB  (total: 9999.4 MB)
```

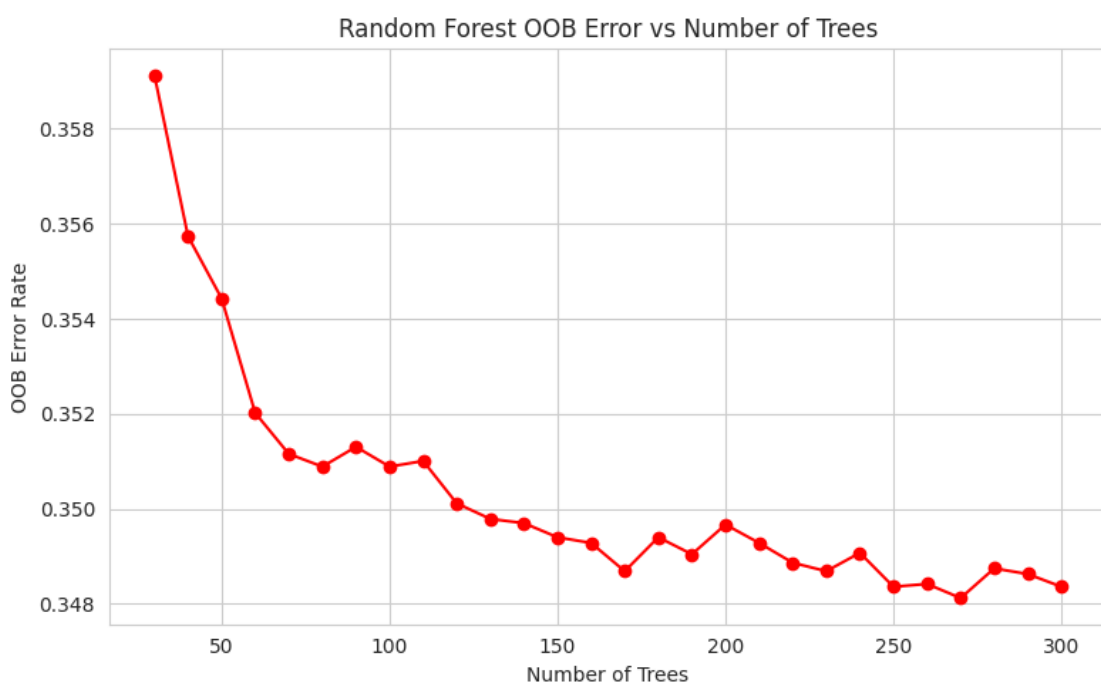
3.2.2 Random Forest Model: Continuous Features

[41]:
```python
# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

from sklearn.utils.class_weight import compute_class_weight
import numpy as np
import warnings
warnings.filterwarnings("ignore")

# Features + target
features = [
    "avg_sentiment_polarity",
    "avg_sentiment_subjectivity",
    "subcategory_rating_score",
    "city_mean_rating"
]

df = restaurant_df[features + ["avg_rating"]].dropna()

X = df[features]
y = df["avg_rating"].round().astype(int)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

#  Compute class weights once (important for warm_start)
classes = np.unique(y_train)
weights = compute_class_weight(
```

```
    class_weight="balanced",
    classes=classes,
    y=y_train
)
class_weights = dict(zip(classes, weights))

# Random Forest for CV
rf = RandomForestClassifier(
    n_estimators=300,
    min_samples_leaf=5,
    class_weight=class_weights,
    n_jobs=-1,
    random_state=42
)

# CV on training set
cv_scores = cross_val_score(rf, X_train, y_train, cv=5, n_jobs=1)
print("CV Accuracy:", round(cv_scores.mean(), 4))
print("CV Std Dev :", round(cv_scores.std(), 4))

#   OOB error curve
print("\nCalculating OOB error curve...")

oob_rf = RandomForestClassifier(
    n_estimators=30,
    min_samples_leaf=5,
    class_weight=class_weights,
    n_jobs=-1,
    random_state=42,
    bootstrap=True,
    oob_score=True,
    warm_start=True
)

oob_errors = []
tree_range = range(30, 301, 10)

for n_trees in tree_range:
    oob_rf.set_params(n_estimators=n_trees)
    oob_rf.fit(X_train, y_train)
    oob_errors.append(1 - oob_rf.oob_score_)

plt.figure(figsize=(8,5))
plt.plot(tree_range, oob_errors, marker="o" , color ='red')
plt.xlabel("Number of Trees")
plt.ylabel("OOB Error Rate")
plt.title("Random Forest OOB Error vs Number of Trees")
```

```python
plt.tight_layout()
plt.show()

# Train final model
print("\nTraining final model...")
rf.fit(X_train, y_train)
print("Done.")

# Predictions
y_pred = rf.predict(X_test)

print("\nTest Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Time & RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
print(f"\nTime elapsed : {time.time() - start_time:.1f}s")
print(f"RAM delta    : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
CV Accuracy: 0.6535
CV Std Dev : 0.0042

Calculating OOB error curve…
```



```
Training final model…
Done.

Test Accuracy: 0.6587

Classification Report:
               precision    recall  f1-score   support

           1       0.50      0.61      0.55       383
           2       0.63      0.62      0.63      1175
           3       0.61      0.64      0.62      1976
           4       0.78      0.70      0.74      3826
           5       0.51      0.60      0.55      1047

    accuracy                           0.66      8407
   macro avg       0.60      0.63      0.62      8407
weighted avg       0.67      0.66      0.66      8407


Confusion Matrix:
 [[ 232  136   13    1    1]
 [ 183  734  230   18   10]
 [  33  258 1269  382   34]
 [   6   36  547 2676  561]
 [   8    9   30  373  627]]

Time elapsed : 115.2s
RAM delta    : 399.2 MB  (total: 10398.6 MB)
```

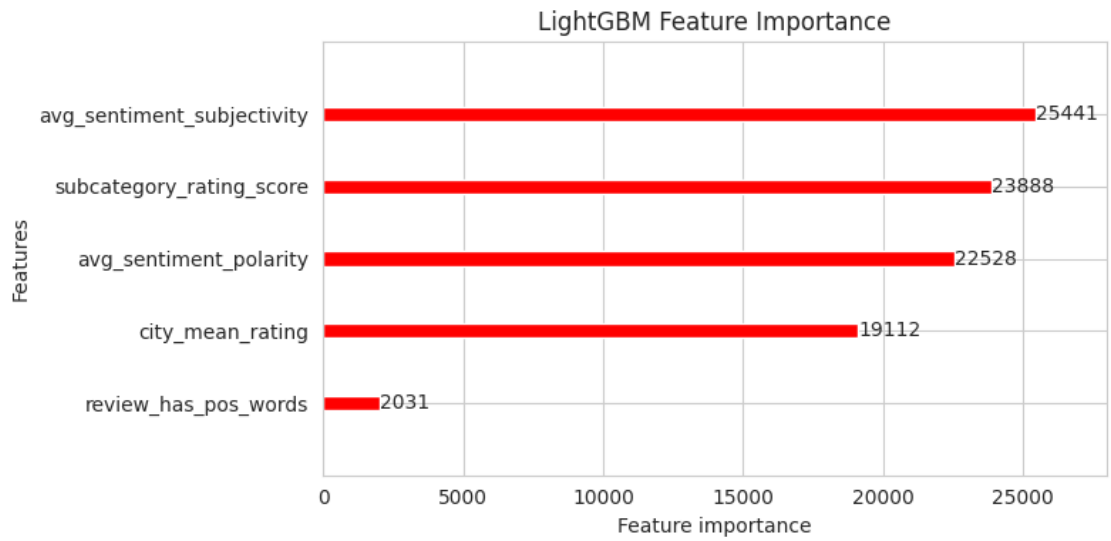
3.2.3 Light Gradient Boosting Machine (LightGBM) Model

[42]:
```python
import lightgbm as lgb

# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

# 1) Features + target (business-level)
print("Step 1/5: Preparing features...")
features = ['review_has_pos_words', 'avg_sentiment_polarity',␣
 ↪'avg_sentiment_subjectivity',
            'subcategory_rating_score', 'city_mean_rating']

df = restaurant_df[features + ['avg_rating']].dropna()
X = df[features]
y = df['avg_rating'].round().astype(int)  # 1-5, matches SVM target
```

```python
# 2) Train/test split
print("Step 2/5: Splitting data...")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3) Model
print("Step 3/5: Defining model...")
lgbm_model = lgb.LGBMClassifier(
    num_leaves=63,
    learning_rate=0.05,
    n_estimators=300,
    subsample=0.8,
    colsample_bytree=0.8,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1,
    verbose=-1
)

# 4) Cross-validation
print("Step 4/5: Running cross-validation...")
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(lgbm_model, X_train, y_train, cv=cv, n_jobs=1)

# 5) Final fit + evaluation
print("Step 5/5: Final fit and evaluation...")
lgbm_model.fit(X_train, y_train)
y_pred = lgbm_model.predict(X_test)

# Results
print(f"\nCV Accuracy : {round(cv_scores.mean(), 4)}")
print(f"CV Std Dev  : {round(cv_scores.std(),  4)}")
print(f"\nTest Accuracy: {round(accuracy_score(y_test, y_pred), 4)}")
print("\nClassification Report:\n", classification_report(y_test, y_pred,␣
 ↪zero_division=0))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Feature importance
lgb.plot_importance(lgbm_model, figsize=(8, 4), title="LightGBM Feature␣
 ↪Importance" , color = 'red')
plt.tight_layout()
plt.show()

# Time & RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
```

```python
print(f"\nTotal time : {time.time() - start_time:.1f}s")
print(f"RAM delta  : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
Step 1/5: Preparing features…
Step 2/5: Splitting data…
Step 3/5: Defining model…
Step 4/5: Running cross-validation…
Step 5/5: Final fit and evaluation…

CV Accuracy : 0.634
CV Std Dev  : 0.0078

Test Accuracy: 0.6347

Classification Report:
               precision    recall  f1-score   support

           1       0.49      0.64      0.56       383
           2       0.61      0.63      0.62      1175
           3       0.59      0.66      0.62      1976
           4       0.80      0.61      0.69      3826
           5       0.46      0.69      0.55      1047

    accuracy                           0.63      8407
   macro avg       0.59      0.65      0.61      8407
weighted avg       0.67      0.63      0.64      8407


Confusion Matrix:
 [[ 245  126    8    0    4]
 [ 194  746  207   15   13]
 [  38  291 1300  312   35]
 [  10   45  641 2320  810]
 [  11   11   30  270  725]]
```




```
Total time : 23.6s
RAM delta  : 4.6 MB  (total: 10406.7 MB)
```

3.2.4 Dense Neural Network Model

[43]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import StandardScaler

# Initialise timer and RAM baseline
start_time = time.time()
ram_before = psutil.Process(os.getpid()).memory_info().rss / 1024**2

# 1) Features + target (business-level - same as LightGBM)
print("Step 1/5: Preparing features...")
features = ['review_has_pos_words', 'avg_sentiment_polarity',␣
 ↪'avg_sentiment_subjectivity',
            'subcategory_rating_score', 'city_mean_rating']

df = restaurant_df[features + ['avg_rating']].copy()
df['city_mean_rating'] = df['city_mean_rating'].fillna(df['city_mean_rating'].
 ↪mean())
df = df.dropna()

X = df[features].values
y = df['avg_rating'].round().astype(int) - 1  # 0-indexed for keras (0-4)
```

```python
# 2) Scale features - DNNs are sensitive to feature scale, tree models are not
print("Step 2/5: Scaling + splitting...")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# 3) Build model
print("Step 3/5: Building model...")
model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(5, activation='softmax')   # 5 classes
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

# 4) Train
print("Step 4/5: Training...")
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# 5) Evaluate
print("Step 5/5: Evaluating...")
y_pred = model.predict(X_test).argmax(axis=1)

# Remap back to 1-5 for readable output matching LightGBM/SVM
y_test_display = y_test + 1
y_pred_display = y_pred + 1

print(f"\nTest Accuracy: {round(accuracy_score(y_test, y_pred), 4)}")
```

```
print("\nClassification Report:\n", classification_report(y_test_display,␣
↪y_pred_display, zero_division=0))
print("\nConfusion Matrix:\n", confusion_matrix(y_test_display, y_pred_display))

# Training curve
plt.figure(figsize=(8, 4))
plt.plot(history.history['accuracy'],     label='Train Accuracy', color =␣
↪'blue')
plt.plot(history.history['val_accuracy'], label='Val Accuracy' , color = 'red')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('DNN Training Curve')
plt.legend()
plt.tight_layout()
plt.show()

# Time & RAM
ram_after = psutil.Process(os.getpid()).memory_info().rss / 1024**2
print(f"\nTotal time : {time.time() - start_time:.1f}s")
print(f"RAM delta  : {ram_after - ram_before:.1f} MB  (total: {ram_after:.1f}␣
↪MB)")
```

```
Step 1/5: Preparing features…
Step 2/5: Scaling + splitting…
Step 3/5: Building model…
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 64)	384
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 5)	165

Total params: 2,629 (10.27 KB)

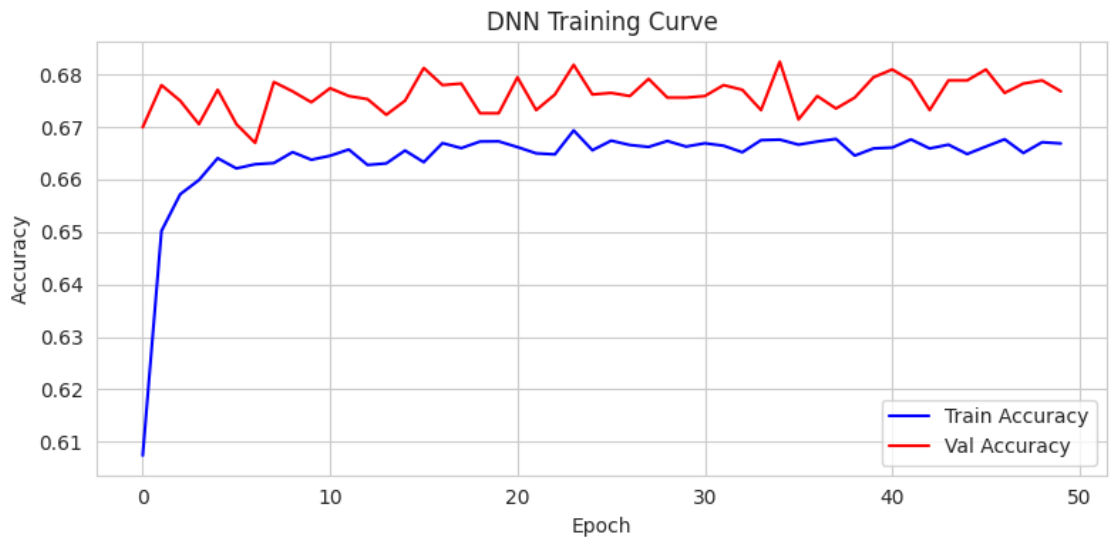
Trainable params: 2,629 (10.27 KB)

```
 Non-trainable params: 0 (0.00 B)

Step 4/5: Training…
Epoch 1/50
946/946              3s 2ms/step -
accuracy: 0.6074 - loss: 0.9676 - val_accuracy: 0.6699 - val_loss: 0.8094
Epoch 2/50
946/946              2s 2ms/step -
accuracy: 0.6502 - loss: 0.8331 - val_accuracy: 0.6780 - val_loss: 0.7921
Epoch 3/50
946/946              2s 2ms/step -
accuracy: 0.6572 - loss: 0.8144 - val_accuracy: 0.6750 - val_loss: 0.7789
Epoch 4/50
946/946              2s 2ms/step -
accuracy: 0.6599 - loss: 0.8055 - val_accuracy: 0.6705 - val_loss: 0.7790
Epoch 5/50
946/946              2s 2ms/step -
accuracy: 0.6640 - loss: 0.7969 - val_accuracy: 0.6771 - val_loss: 0.7749
Epoch 6/50
946/946              3s 2ms/step -
accuracy: 0.6621 - loss: 0.7933 - val_accuracy: 0.6705 - val_loss: 0.7738
Epoch 7/50
946/946              3s 2ms/step -
accuracy: 0.6629 - loss: 0.7951 - val_accuracy: 0.6670 - val_loss: 0.7713
Epoch 8/50
946/946              2s 2ms/step -
accuracy: 0.6631 - loss: 0.7901 - val_accuracy: 0.6786 - val_loss: 0.7687
Epoch 9/50
946/946              2s 2ms/step -
accuracy: 0.6652 - loss: 0.7888 - val_accuracy: 0.6768 - val_loss: 0.7701
Epoch 10/50
946/946              2s 2ms/step -
accuracy: 0.6637 - loss: 0.7845 - val_accuracy: 0.6747 - val_loss: 0.7699
Epoch 11/50
946/946              2s 2ms/step -
accuracy: 0.6645 - loss: 0.7856 - val_accuracy: 0.6774 - val_loss: 0.7665
Epoch 12/50
946/946              3s 2ms/step -
accuracy: 0.6657 - loss: 0.7819 - val_accuracy: 0.6759 - val_loss: 0.7693
Epoch 13/50
946/946              2s 2ms/step -
accuracy: 0.6628 - loss: 0.7862 - val_accuracy: 0.6753 - val_loss: 0.7666
Epoch 14/50
946/946              2s 2ms/step -
accuracy: 0.6631 - loss: 0.7827 - val_accuracy: 0.6723 - val_loss: 0.7682
Epoch 15/50
946/946              2s 2ms/step -
```

```
accuracy: 0.6655 - loss: 0.7825 - val_accuracy: 0.6750 - val_loss: 0.7670
Epoch 16/50
946/946            2s 2ms/step -
accuracy: 0.6633 - loss: 0.7824 - val_accuracy: 0.6812 - val_loss: 0.7669
Epoch 17/50
946/946            2s 2ms/step -
accuracy: 0.6669 - loss: 0.7816 - val_accuracy: 0.6780 - val_loss: 0.7667
Epoch 18/50
946/946            2s 2ms/step -
accuracy: 0.6660 - loss: 0.7782 - val_accuracy: 0.6783 - val_loss: 0.7665
Epoch 19/50
946/946            2s 2ms/step -
accuracy: 0.6672 - loss: 0.7808 - val_accuracy: 0.6726 - val_loss: 0.7665
Epoch 20/50
946/946            2s 2ms/step -
accuracy: 0.6673 - loss: 0.7796 - val_accuracy: 0.6726 - val_loss: 0.7673
Epoch 21/50
946/946            2s 2ms/step -
accuracy: 0.6662 - loss: 0.7787 - val_accuracy: 0.6795 - val_loss: 0.7686
Epoch 22/50
946/946            2s 2ms/step -
accuracy: 0.6650 - loss: 0.7792 - val_accuracy: 0.6732 - val_loss: 0.7672
Epoch 23/50
946/946            2s 2ms/step -
accuracy: 0.6648 - loss: 0.7802 - val_accuracy: 0.6762 - val_loss: 0.7671
Epoch 24/50
946/946            2s 2ms/step -
accuracy: 0.6693 - loss: 0.7788 - val_accuracy: 0.6818 - val_loss: 0.7678
Epoch 25/50
946/946            2s 2ms/step -
accuracy: 0.6656 - loss: 0.7789 - val_accuracy: 0.6762 - val_loss: 0.7660
Epoch 26/50
946/946            2s 2ms/step -
accuracy: 0.6674 - loss: 0.7797 - val_accuracy: 0.6765 - val_loss: 0.7687
Epoch 27/50
946/946            2s 2ms/step -
accuracy: 0.6666 - loss: 0.7785 - val_accuracy: 0.6759 - val_loss: 0.7694
Epoch 28/50
946/946            2s 2ms/step -
accuracy: 0.6662 - loss: 0.7780 - val_accuracy: 0.6792 - val_loss: 0.7660
Epoch 29/50
946/946            2s 2ms/step -
accuracy: 0.6673 - loss: 0.7785 - val_accuracy: 0.6756 - val_loss: 0.7678
Epoch 30/50
946/946            2s 2ms/step -
accuracy: 0.6663 - loss: 0.7767 - val_accuracy: 0.6756 - val_loss: 0.7689
Epoch 31/50
946/946            2s 2ms/step -
```

```
accuracy: 0.6669 - loss: 0.7795 - val_accuracy: 0.6759 - val_loss: 0.7669
Epoch 32/50
946/946            2s 2ms/step -
accuracy: 0.6664 - loss: 0.7758 - val_accuracy: 0.6780 - val_loss: 0.7668
Epoch 33/50
946/946            2s 2ms/step -
accuracy: 0.6652 - loss: 0.7782 - val_accuracy: 0.6771 - val_loss: 0.7660
Epoch 34/50
946/946            2s 2ms/step -
accuracy: 0.6675 - loss: 0.7773 - val_accuracy: 0.6732 - val_loss: 0.7689
Epoch 35/50
946/946            2s 2ms/step -
accuracy: 0.6676 - loss: 0.7760 - val_accuracy: 0.6824 - val_loss: 0.7701
Epoch 36/50
946/946            2s 2ms/step -
accuracy: 0.6666 - loss: 0.7775 - val_accuracy: 0.6714 - val_loss: 0.7676
Epoch 37/50
946/946            2s 2ms/step -
accuracy: 0.6672 - loss: 0.7752 - val_accuracy: 0.6759 - val_loss: 0.7686
Epoch 38/50
946/946            2s 2ms/step -
accuracy: 0.6677 - loss: 0.7757 - val_accuracy: 0.6735 - val_loss: 0.7691
Epoch 39/50
946/946            2s 2ms/step -
accuracy: 0.6645 - loss: 0.7795 - val_accuracy: 0.6756 - val_loss: 0.7692
Epoch 40/50
946/946            2s 2ms/step -
accuracy: 0.6659 - loss: 0.7767 - val_accuracy: 0.6795 - val_loss: 0.7687
Epoch 41/50
946/946            2s 2ms/step -
accuracy: 0.6661 - loss: 0.7765 - val_accuracy: 0.6809 - val_loss: 0.7665
Epoch 42/50
946/946            2s 2ms/step -
accuracy: 0.6676 - loss: 0.7765 - val_accuracy: 0.6789 - val_loss: 0.7684
Epoch 43/50
946/946            2s 2ms/step -
accuracy: 0.6659 - loss: 0.7775 - val_accuracy: 0.6732 - val_loss: 0.7682
Epoch 44/50
946/946            2s 2ms/step -
accuracy: 0.6666 - loss: 0.7771 - val_accuracy: 0.6789 - val_loss: 0.7674
Epoch 45/50
946/946            2s 2ms/step -
accuracy: 0.6648 - loss: 0.7768 - val_accuracy: 0.6789 - val_loss: 0.7686
Epoch 46/50
946/946            2s 2ms/step -
accuracy: 0.6663 - loss: 0.7778 - val_accuracy: 0.6809 - val_loss: 0.7673
Epoch 47/50
946/946            2s 2ms/step -
```

```
accuracy: 0.6676 - loss: 0.7761 - val_accuracy: 0.6765 - val_loss: 0.7673
Epoch 48/50
946/946             2s 2ms/step -
accuracy: 0.6650 - loss: 0.7775 - val_accuracy: 0.6783 - val_loss: 0.7702
Epoch 49/50
946/946             2s 2ms/step -
accuracy: 0.6671 - loss: 0.7747 - val_accuracy: 0.6789 - val_loss: 0.7657
Epoch 50/50
946/946             2s 2ms/step -
accuracy: 0.6669 - loss: 0.7752 - val_accuracy: 0.6768 - val_loss: 0.7668
Step 5/5: Evaluating…
263/263             0s 1ms/step

Test Accuracy: 0.6841

Classification Report:
               precision    recall  f1-score   support

           1       0.68      0.43      0.53       383
           2       0.62      0.73      0.67      1175
           3       0.67      0.54      0.60      1976
           4       0.72      0.85      0.78      3826
           5       0.63      0.40      0.49      1047

    accuracy                           0.68      8407
   macro avg       0.66      0.59      0.61      8407
weighted avg       0.68      0.68      0.67      8407


Confusion Matrix:
 [[ 166  209    4    4    0]
 [  56  859  211   44    5]
 [  14  261 1063  631    7]
 [   2   42  305 3247  230]
 [   6   14   13  598  416]]
```




```
Total time : 91.5s
RAM delta  : -342.4 MB  (total: 10064.3 MB)
```

[43]:

```
```